

Bachelor's Thesis in Information Systems

Differentiable Simulation for Efficient and Versatile Quadruped Locomotion

Differenzierbare Simulation für effiziente und vielseitige Fortbewegung von Vierbeinern

Supervisor	Prof. Dr.-Ing. habil. Alois C. Knoll
Advisor	Yulong Xiao, M.Sc.
Author	Jakub Jura
Date	September 21, 2026 in Munich

Disclaimer

I confirm that this Bachelor's Thesis is my own work and I have documented all sources and material used.

Munich, September 21, 2026



(Jakub Jura)

Abstract

This thesis investigates differentiable simulation for quadruped locomotion on challenging terrain. The work extends an existing differentiable locomotion framework for the Unitree Go2 by introducing full-pipeline vectorization, a fused CUDA kernel with an analytical adjoint, differentiable terrain height sampling, privileged height observations, and a simulated depth camera. The resulting implementation enables large-scale training, providing up to 12x speed-up over the inherited code. Further experiments showed that neither terrain gradients nor terrain observations yielded a reliable improvement in hard-terrain locomotion. Across the training runs, the terrain curriculum consistently collapsed to its easiest level, limiting the policies' ability to learn terrain-dependent behavior, mainly due to instability under omnidirectional commands and the absence of a direct penalty for falling. Four interventions were evaluated, but none prevented the collapse.

Kurzfassung

Diese Arbeit untersucht die differenzierbare Simulation der Fortbewegung vierbeiniger Roboter in anspruchsvollem Gelände. Die Arbeit erweitert ein bestehendes Framework für die differenzierbare Fortbewegung des Unitree Go2 durch die Einführung einer vollständigen Vektorisierung der Pipeline, eines fusionierten CUDA-Kernels mit analytisch berechnetem adjungiertem Rückwärtspass, differenzierbarer Geländehöhenabtastung, privilegierter Höhenbeobachtungen sowie einer simulierten Tiefenkamera. Die resultierende Implementierung ermöglicht ein Training im großen Maßstab und erzielt eine bis zu 12-fache Beschleunigung gegenüber dem ursprünglichen Code. Weitere Experimente zeigten, dass weder Geländegradien noch Geländebeobachtungen zu einer zuverlässigen Verbesserung der Fortbewegung in schwierigem Gelände führten. Über die Trainingsläufe hinweg fiel das Gelände-Curriculum konsistent auf die einfachste Stufe zurück, was die Fähigkeit der Policies einschränkte, geländeabhängiges Verhalten zu erlernen – hauptsächlich aufgrund von Instabilität bei omnidirektionalen Steuerbefehlen und dem Fehlen einer direkten Bestrafung für Stürze. Es wurden vier Maßnahmen evaluiert, doch keine konnte diesen Rückfall verhindern.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement and Research Questions	2
1.3	Overview of the Approach	3
1.4	Contributions	4
1.4.1	Systems and Implementation Contributions	4
1.4.2	Locomotion Method Contributions	5
2	Related Work	7
2.1	Model-Based Quadruped Control	7
2.2	Learning-Based Quadruped Locomotion	9
2.3	Analytic Policy Gradients and Differentiable Simulation as a Learning Framework	11
2.4	Perception for Locomotion	13
3	Method - Baseline, Terrain Perception and Structural Limits	15
3.1	The Differentiable SRBD State Alignment Method	15
3.1.1	Centroidal Dynamics of the SRBD Surrogate	15
3.1.2	Mapping Policy Outputs to Ground Reaction Forces	17
3.1.3	The State Alignment Coupling Formulation	17
3.2	The Forked Implementation	18
3.3	Differentiable Terrain Height Sampling	18
3.4	Depth Camera Vision Policy	19
3.5	Loss Design	19
3.6	Structural Limits	20
3.6.1	Limit 1: No Perceptual Output Channel	21
3.6.2	Limit 2: Truncated Gradient Horizon	21
3.6.3	Limit 3: Termination Outside of the Gradient	21
3.7	Interventions on the Structural Limits	21
3.7.1	Foothold Correction Channel	21
3.7.2	Extended Gradient Horizon	22
3.7.3	Differentiable Termination Surrogate	22
4	Implementation - Vectorization and GPU Acceleration	23
4.1	Software Architecture and Design Patterns	23
4.2	Pipeline Vectorization and Batching	24
4.3	Fused CUDA SRBD Kernel and Analytic Adjoint	24
4.3.1	Fused Forward Kernel Design	25
4.3.2	Analytical Adjoint Backward Pass	25
4.3.3	Profiling	26
4.4	GPU Perception Stack	26
4.4.1	Ray Casting Depth Camera	26

4.4.2	The Empirical Reproducibility Floor	27
5	Experiments and Results	29
5.1	Experimental Setup and Statistical Criteria	29
5.1.1	Robot and Simulator Parameters	29
5.1.2	The Procedural Terrain Curriculum	29
5.1.3	Statistical Criteria	30
5.2	Baseline Reproduction and Parity	30
5.3	RQ1 - Training Throughput Results	31
5.3.1	Speed-up Compared to the Baseline	31
5.3.2	Throughput Scaling	32
5.3.3	The Core Performance Takeaway	33
5.4	RQ2 - The Terrain Information Study	33
5.4.1	Experiment	33
5.4.2	Results	34
5.5	Perceptual Saliency Diagnostics	35
5.5.1	Comparison Before the Corruption Test	36
5.5.2	The Corruption Matrix	36
5.6	Why the Curriculum Collapses	36
5.6.1	The Shape of the Collapse	37
5.6.2	The Promotion Arithmetic	37
5.6.3	Omnidirectional Commands on Flat Ground	37
5.6.4	Why the Objective Permits Tilt	38
5.7	RQ3 - The Four Interventions	38
5.7.1	Intervention 1: Differentiable Termination Surrogate	38
5.7.2	Intervention 2: Terrain Aware Swing Planner	39
5.7.3	Intervention 3: Explicit Foothold Residual	40
5.7.4	Intervention 4: Extended Gradient Horizon	40
6	Conclusions and Future Work	41
6.1	Conclusions	41
6.2	Future Work	42
6.3	Limitations	43
A	Code and Results	45
	Bibliography	47

Chapter 1

Introduction

1.1 Motivation

The development of robots able to perform in hard and active environments is one of the main challenges in robotics [Ble+18a; Hwa+19; Lee+20]. Wheeled or tracked robot systems achieve high efficiency and speed. They have, however, very limited abilities to navigate over steep inclines, physical obstacles, and terrain discontinuities [Rai86; Hut+16; Lee+20; Fan+18]. In the natural world, the legged organisms handle such environments without problems. This causes legged robots to appeal as a very interesting alternative to their wheeled counterparts. By utilizing discrete footholds, legged robots can isolate their main body from terrain irregularities. This enables agile mobility across steps, stairs, gaps, and steep slopes [Rai86; Hut+16; Lee+20; Fan+18].

Control of such legged robots is, unfortunately, very hard. It poses computational complications due to multidimensional state spaces and non-smooth contact physics. Additionally, it needs to operate within strict torque and kinematic limits [Di +18; Suh+22]. In response to these challenges, two primary control paradigms have historically emerged.

The first is model-based optimal control. It is often implemented as Model Predictive Control (MPC) combined with Whole Body Control (WBC) [Di +18; Kim+19]. Such methods typically use simplified physical models to optimize contact forces and joint trajectories [Di +18; Kim+19]. Although model-based control provides strong interpretability and safety, it requires substantial computational capabilities and significant efforts in manual modeling of such policies [Di +18].

The second paradigm is model-free Reinforcement Learning (RL). This approach demonstrated high agility by training in massively parallel simulation environments [Hwa+19; Lee+20; Rud+22; Mar+22]. But its potentialities come at a price of very high sample inefficiency. Training of neural network policies with the help of RL requires millions or even billions of simulated steps [Hwa+19; Rud+22; SKS24]. This comes from the fact that RL treats the physical environment as a black box and relies on zeroth-order optimization [SKS24; Suh+22]. On top of that, RL training often depends on manual reward engineering to shape correct behaviors and prevent policy collapse. Otherwise, it could lead to unnatural gaits [Hwa+19; Pen+18a].

Differentiable simulation has emerged as an attractive third alternative. It attempts to combine the physical grounding of model-based approaches with the learning capabilities of RL [HSS25; Xu+22; SKS24]. The physics engine is treated as a differentiable operator allowing analytical policy gradients to be backpropagated directly from the task loss through the simulated rollout to the policy's network weights [Xu+22; SKS24]. In theory, such first-order gradient information should substantially reduce sample complexity with elimination of trial-and-error exploration [HSS25; Xu+22; SKS24]. However, Newbury et al. [New+24]

described that application of analytical gradients into policy learning is impeded by the non-smoothness of contact dynamics and issues related to gradient explosion and vanishing. Also, the selection of the differentiation pipeline (automatic differentiation vs. source-code transformation) plays a role [Suh+22; Met+22].

1.2 Problem Statement and Research Questions

While differentiable simulators offer the potential for highly efficient policy optimization, their use in legged locomotion is limited by the aforementioned mathematical challenges [Suh+22; Met+22]. Non-smooth contact mechanics results in gradients that are nearly always zero or highly variable. Moreover, backpropagation of gradients over longer trajectories results in divergence of policy optimization or freezing in poor local minima [Met+22].

To address these limits, Song et al. [SKS24] avoid differentiation directly in the contact-rich simulator. Instead, at each timestep, two simulations run in parallel. One is a full-fidelity, non-differentiable simulator (NVIDIA IsaacGym [Mak+21]), providing the ground-truth physical state. Second is a simplified Single Rigid Body Dynamics (SRBD) model serving as a differentiable surrogate. They are coupled together at each step by aligning the state value with ground truth and maintaining a scaled gradient pathway through the surrogate. Consequently, backpropagation through time is confined to a short, smooth window rather than traversing contact events. This prevents the exponential growth of gradient variance. This shares conceptual similarities with short-horizon actor-critic architectures [Xu+22], which restrict gradient propagation to short time windows and use a value critic to bootstrap future returns.

This hybrid formulation successfully demonstrates efficient learning of proprioceptive gaits on flat ground. However, its application to a hard curriculum and the use of exteroceptive signals were not investigated [SKS24]. Such signals can make the loss less smooth, for example, when a foot gets stuck against an obstacle [Suh+22; Lee+20]. Reinforcement learning approaches address this challenge through height-scan observations or online adaptation [Che+23; Aga+22; Kum+21]. First-order policy gradients have been applied to perceptive quadruped control [Luo+24], but only through a critic-bootstrapped horizon on a general articulated simulator for tasks such as obstacle avoidance and local navigation, with vision incorporated as an observation. To author’s knowledge, routing terrain geometry directly into the analytical gradients of a differentiable simulator has not been studied. These gaps lead to three research questions:

Research Question 1 (RQ1)

What determines training throughput in a differentiable simulation locomotion pipeline, and is a hand-written fused CUDA kernel necessary to achieve it?

Research Question 2 (RQ2)

To what extent does propagating terrain gradients through a differentiable SRBD surrogate improve hard-terrain locomotion, relative to (i) a blind baseline and (ii) the identical terrain signal supplied only as a policy observation?

Research Question 3 (RQ3)

Which structural properties of the reference-tracking formulation limit the actionability of terrain gradients, and can targeted architectural interventions make them actionable?

1.3 Overview of the Approach

The system integrates a handwritten, differentiable physical surrogate with NVIDIA's full-physics simulator [SKS24]. At each timestep (2 ms), NVIDIA Isaac Gym [Mak+21] advances B parallel Go2 robots on the GPU, handling contact dynamics, PD joint control, and collision meshes. Simultaneously, a simplified SRBD model re-integrates the same estimated ground reaction forces, evaluated either in PyTorch or by a fused CUDA kernel.

At each step, the two simulators are coupled using the (1.1) formula. This sets the state value to the ground truth Isaac Gym measurement while retaining the analytical gradient corridor across steps [SKS24].

$$x = x_{isaac} + \alpha \cdot (x_{srbd} - \text{detach}(x_{srbd})) \quad (1.1)$$

Numerically, the bracket evaluates to zero, resulting in a forward rollout that matches the full-fidelity trajectory. During differentiation, the bracket contributes $\alpha \cdot \partial x_{srbd} / \partial \theta$, so the backward pass observes only the smooth analytic dynamics. Here $\alpha = 0.9$ is set by default. Single optimizer iteration differentiates a 24-step window, which corresponds to 48 ms of simulated time. After this time, the surrogate state is detached, and the computational graph is reset.

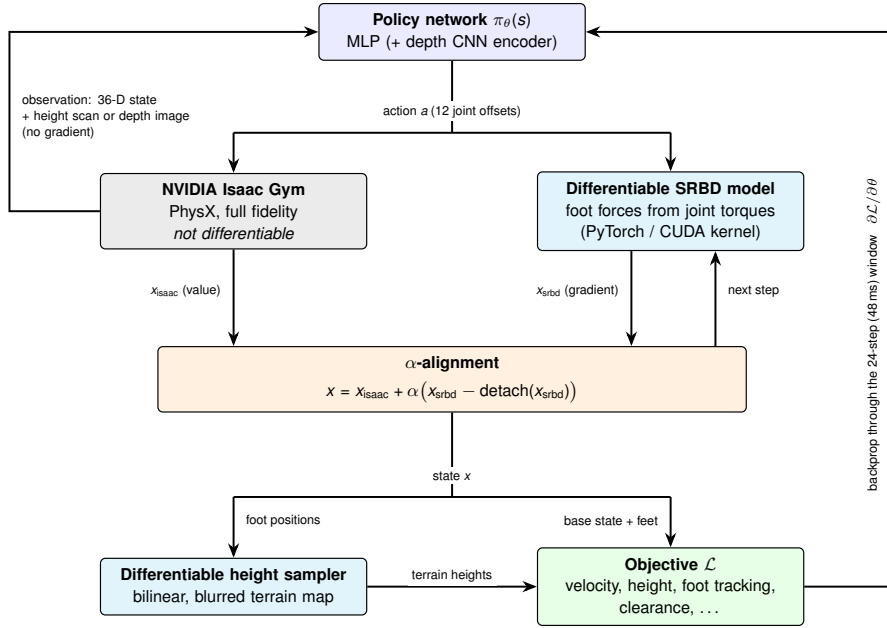


Figure 1.1: Schematic overview of single training step. The policy action drives both simulators. Their outputs are then fused. The terrain signal enters through the loss or through the observation. The height sampler is queried at SRBD's predicted positions, so the terrain slope can be backpropagated through the surrogate into the policy weights. The differentiable height sampler works only when the terrain signal is routed through the loss.

Terrain perception is integrated into the pipeline. It can either come as a gradient from the terrain slope or as an observation, being part of the policy input. The height sampler indicates the elevation of the ground at the queried point. Sampling a local terrain-height grid around the robot is commonly used in RL approaches [Rud+22]. What is new here is that the lookup is differentiable. It interpolates between the four nearest points rather than taking the closest one. Therefore, the height changes smoothly as the query point moves. This allows computing a derivative that represents the local terrain slope, thereby letting

that slope reach the policy. It is queried at the base and foot positions predicted by the SRBD surrogate. The sampled heights are routed directly into the objective function via a terrain-relative height penalty and a swing-foot clearance loss. The privileged variant further appends a 187-point yaw-aligned height scan to the observation, following the layout of Rudin et al. [Rud+22].

To assess whether deployable perception can also handle hard terrain, a forward-facing depth camera is simulated with NVIDIA Warp [NVI26]. It produces a depth image that is processed by a small convolutional neural network (CNN) encoder, following the approach of Dong et al. [Don+26]. The depth image itself does not carry a gradient. However, the encoder’s weights are trained end-to-end, since the chain from its output through the action and surrogate dynamics to the loss remains differentiable.

1.4 Contributions

This thesis focuses on parallelization, acceleration, extension, and evaluation of a differentiable simulation locomotion framework for the Unitree Go2 quadruped. The codebase is a fork of a third-party implementation [Zha26] of Song et al.’s work [SKS24]. It already existed at the Chair of Robotics, Artificial Intelligence and Real-Time Systems at TUM. The contributions of this thesis are grouped into systems/implementations contributions and locomotion method contributions. This reflects two axes along which the baseline was extended.

1.4.1 Systems and Implementation Contributions

Full pipeline vectorization

All per environment Python loops were eliminated across the codebase. These loops were replaced by fully batched GPU tensor operations. This allows to run simulations with thousands of robots simultaneously.

Fused CUDA Kernel for Analytical Adjoint Computation

A fused forward CUDA kernel was developed for SRBD dynamics. It was paired with a custom analytical adjoint backward pass, replacing PyTorch autograd’s operation-by-operation decomposition of that step with a single node.

Parity and validation suites

A CPU/GPU test suite to verify the correctness of the new components was implemented. Four test groups check the CUDA forward and backward passes, the batched implementation against the original version, gradients through the perception module, and the vision-based policy. Four additional tests verify the experimental modifications. They check whether the tilt barrier has the correct sign, the foothold loss produces the expected gradient near a stair edge, and both new loss terms become identical to the original objective when disabled, and terrain type is independent of the command distribution.

Throughput profiling and decomposition

Detailed profiling and throughput decomposition were conducted. Additionally, the project’s bottlenecks were measured to answer RQ1. It was shown that batching outperforms custom accelerators because advanced physics simulators account for most of the runtime.

Curriculum terrain port

The dynamic hard terrain curriculum of `legged_gym` [Rud+22] was integrated into the differentiable framework. It matches the `legged_gym`'s command ranges and episode timeouts.

1.4.2 Locomotion Method Contributions**Differentiable terrain gradient sampler**

A yaw-aligned bilinear height field sampler was developed. It computes both exact and Gaussian-blurred height fields, routing the terrain slope and swing-clearance gradients directly into the policy.

End-to-end perceptive policy

An NVIDIA Warp ray casting depth camera and a convolutional encoder were integrated into the differentiable rollout. The depth image remains a gradient-free constant. The encoder's weights are trained by the task loss. The chain from the encoder's output carries the gradient through the action and the differentiable SRBD dynamics into the loss. The training signal for the vision encoder is thus an analytic derivative of a physics model rather than a sampled return or a distillation target [Don+26; Zha+25].

Structural limits analysis

An analysis of the current architecture and its abilities to handle terrain information was proposed.

Four controlled interventions

Four targeted interventions were developed to overcome potential structural limitations of the framework. The soft tilt barrier, the foothold landing height, the foothold residual corrections, and the extended rollout horizon. It was completed with rigorous logging mechanisms.

Perceptual saliency diagnostic

A test-time exteroceptive input corruption test was proposed to assess whether the policy leverages terrain information. It works by zeroing or substituting the terrain scan.

Chapter 2

Related Work

2.1 Model-Based Quadruped Control

The development of dynamic legged locomotion has predominantly relied on model-based control theory, which models the physical system with deterministic equations of motion and optimizes trajectories over a receding horizon [Kum+21]. The foundational principles of dynamic balance in legged robots were established by Marc Raibert, who introduced a three-part control system: hopping height control via stance-leg thrust, forward running velocity control via foot placement, and body attitude control via hip torques during stance [Rai86; Rai+83]. This decoupled control paradigm demonstrated that stable, highly dynamic gaits such as trotting, bounding, and pacing can be achieved through relatively simple feedback laws, modeling the robot as a planar or three-dimensional inverted pendulum with a springy leg [Rai86; Rai+83]. The stabilizing effect of foot placement was later analyzed within the Linear Inverted Pendulum (LIP) framework. In this context, describing the next foothold as a linear function of body velocity is provably stable within a bounded range of the velocity feedback coefficient, and this stable parameter corridor expands monotonically as the stepping frequency increases. Both the Raibert touchdown heuristic and the capture point are specific instances of this model [Ye+22].

As legged robotic platforms advanced to multi-degree-of-freedom (DoF) architectures with complex kinematics and torque-controlled actuators, such as the ANYmal [Hut+16] and the MIT Cheetah series [Ble+18a], classical decoupled heuristics became inadequate for stabilizing high-speed, multi-body interactions [Hwa+19]. As a result, model-based optimal control approaches divided the control architecture into two principal layers. The first layer is a high-level Model Predictive Control (MPC) planner that determines the global trajectory of the robot's main body over a receding horizon. The second layer is a low-level Whole-Body Control (WBC) or Whole-Body Impulse Control (WBIC) tracker, which translates the recommended contact forces into joint torques while satisfying physical inequality constraints [Pes+25; Kim+19; Hut+16; Di+18].

The high-level MPC planner relies heavily on reduced-order physical abstractions. One such widely utilized abstraction is the aforementioned Single Rigid Body Dynamics (SRBD) model, which projects the entire mass of the robot into a single rigid body at the center of mass (CoM) and treats the legs as mass-free force-actuators that apply point-contact forces at the ground-reaction interfaces [Pes+25; Di+18; Kim+19]. This simplification abstracts away the complex and high-frequency internal dynamics of the joints and isolates the centroidal momentum equations [OGL13; Di+18]. Formally, the Newton-Euler equations of motion governed by the SRBD assumption project the linear and angular accelerations as functions of the contact forces $\mathbf{f}_i$:

$$\sum_{i=1}^{n_c} \mathbf{f}_i + m\mathbf{g} = m\ddot{\mathbf{x}}_{CoM} \quad (2.1)$$

$$\sum_{i=1}^{n_c} (\mathbf{p}_i - \mathbf{x}_{CoM}) \times \mathbf{f}_i = \frac{d}{dt}(\mathbf{I}\boldsymbol{\omega}) \quad (2.2)$$

where $\mathbf{p}_i$ represents the position of the i -th foot, $\mathbf{I}$ is the body inertia tensor, and $\mathbf{g}$ is the gravity vector [Di + 18; Kim+19]. These equations represent the core physics reproduced by the analytical surrogate used in this project [SKS24]. Solving such equations over an extended horizon is computationally intensive due to the non-linear nature of angular dynamics. To speed up computation, current methods simplify the robot's rotational dynamics by assuming small roll and pitch angles and by using a reference orientation. This significantly reduces the computational cost and allows the problem to be solved as a convex quadratic program (QP) [Pes+25; Di + 18; Kim+19].

These simplifications are introduced to make the optimization problem easier to solve. They are not part of the dynamics model itself. The convex QP works correctly only when the robot remains approximately level. The resulting controller is computationally effective but has limited capability to adapt footstep timing and placement online. In contrast, the underlying physical equations (SRBD) naturally describe three-dimensional body motion, including vertical CoM motion and arbitrary body orientations, and can be combined with contact locations on uneven three-dimensional terrain [Pes+25; Di + 18; Kim+19; Kui+16]. This distinction is significant for the present work, which directly differentiates the SRBD equations rather than applying convexification [SKS24].

To apply these contact forces on physical hardware, model-based optimal control pairs the convex MPC planner with a whole-body joint-space controller. The whole-body controller solves a cascade of QP optimizations at high frequency, matching joint torques, accelerations, and contact forces to the target CoM accelerations while satisfying friction-cone inequality constraints, joint torque limits, and the rigid body kinematics of the multi-body system [Kim+19; Hut+16; Kui+16]. These multi-body kinematics are defined using spatial vector algebra, which combines three-dimensional rotational and translational quantities into six-dimensional representations to recursively propagate positions, velocities, and accelerations along the kinematic tree [Kim+19; OGL13]. In modern WBC architectures, the low-level torque tracking is governed by articulated-body algorithms or recursive Newton-Euler formulations, which map contact impulses and joint motion axes to the physical actuators [Kim+19; New+24].

Despite the mathematical rigor and demonstrated success of model-based MPC-WBC architectures, they present fundamental limitations. First, these approaches require substantial manual engineering of gait-transition structures and foothold-selection heuristics [Win+18; Yan+21; Lee+20]. Because the contact sequence (such as trotting gait phases) is typically predetermined, the robot cannot actively adjust its cadence or transition to emergent gaits when encountering unexpected obstacles or slippery surfaces [Win+18]. Trajectory Optimization frameworks such as TOWR [Win+18] parameterize foot motion in phases and can optimize footholds, contact sequences, and phase durations simultaneously. However, the resulting non-linear programs are prone to poor local minima and are generally too slow for reactive online execution. Online non-linear formulations sacrifice some generality to achieve real-time performance [Bel+18].

Second, model-based controllers face challenges in exteroceptive terrain coupling. In standard implementations, foothold selection is determined using a nominal planar rule, which is subsequently adjusted based on a local terrain height field by a separate, largely decoupled heuristic layer [Jen+22; Fan+18; Mik+22]. When the local height map is noisy or

contains discontinuous features, such as stairs, the decoupling between the force-optimizing MPC and the foot-positioning heuristic can lead to failures, such as swing feet striking the vertical riser of a step [Ble+18b; Jen+22]. The standard mitigation strategy is to detect early or late contacts and treat them as events rather than adhering to a fixed time schedule [Ble+18b; Di+18].

This thesis does not discard the decoupled heuristics of classical control. The reproduced baseline plans footholds with a Raibert rule and schedules contacts from a fixed gait phase [SKS24]. The contribution aims instead to open a path for terrain into the differentiated objective. Whether a reference-tracking objective built on those inherited heuristics can act on such a signal at all.

2.2 Learning-Based Quadruped Locomotion

In response to the heavy manual modeling effort and sensitivity to modeling errors characteristic of model-based frameworks, reinforcement learning (RL) has emerged as a powerful alternative for synthesizing robust legged controllers [Lee+20; Hwa+19]. Rather than explicitly modeling every aspect of the control dynamics, RL formulates the control problem as a Markov Decision Process (MDP), enabling an agent to discover highly agile and adaptive motor skills through trial-and-error interactions in simulated environments [Lee+20; Hwa+19].

The early application of RL to legged systems was severely bottlenecked by sample inefficiency, the high cost of collecting real-world interaction data, and the reality gap (sim-to-real transfer) [Hwa+19]. Training directly on physical robots was particularly challenging and expensive for dynamically balancing systems. Traditional RL algorithms required a huge number of physical interactions. This increased the risk of mechanical failure and limited training to controlled laboratory environments [Hwa+19]. This changed with the adoption of deep neural network function approximators and massively parallel simulation engines. The development of NVIDIA Isaac Gym [Mak+21] enabled the simultaneous simulation of thousands of parallel robot environments entirely on a single GPU [Rud+22]. Both the physics state and policy inference could remain on the GPU. This removed costly CPU-to-GPU data transfers from the training loop. As a result, policies could experience billions of simulated state transitions within minutes of wall-clock time [Mak+21; Rud+22].

Using this parallel simulation architecture, Rudin et al. [Rud+22] showed that robust locomotion policies for the ANYmal robot could be trained in minutes. Training on flat terrain took less than four minutes. Training on uneven terrain took around twenty minutes on a single GPU [Rud+22]. The method uses Proximal Policy Optimization (PPO) [Sch+17b]. PPO was developed as a simpler alternative to trust-region methods such as TRPO [Sch+17a]. Rudin et al. combine PPO with a game-inspired curriculum that gradually increases terrain difficulty [Rud+22]. The policy maps proprioceptive observations directly to target joint positions. Low-level PD controllers then track these target positions [Rud+22].

PPO maximizes the clipped surrogate objective

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (2.3)$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the probability ratio between the new and old policies [Sch+17b].

The term $\hat{A}_t$ represents the estimated advantage, while ϵ controls the clipping range. The clipping limits how far the new policy can move from the old policy during a single update [Sch+17b; Rud+22]. The policy is trained using a multi-term reward function. It encourages the robot to track the desired linear and angular velocities. It also penalizes undesirable

behaviors. These include excessive joint torques, large joint accelerations, base roll and pitch deviations, and foot slipping [Rud+22].

To reduce the burden of manually designing reward functions for highly dynamic skills, researchers introduced example-guided RL [Pen+18a]. DeepMimic [Pen+18a] uses reference motion data to guide the learning process. It adds a motion-tracking objective to the reward function. The policy then learns to reproduce the demonstrated motion in a physics-based simulation. The method was used to learn highly dynamic skills, including flips and spins. A similar motion-mimicking problem was later formulated using differentiable simulation. DiffMimic [Ren+23] replaces RL-based policy optimization with optimization based on gradients through a differentiable simulator. This turns motion mimicking into a state-matching problem. The method achieves faster, more sample-efficient learning than RL-based approaches [Ren+23]. For example, DiffMimic learns a backflip in about ten minutes. It can learn to perform the motion cyclically in about three hours, compared with roughly one day for DeepMimic [Ren+23]. This illustrates the sample-efficiency advantage.

When deploying simulated policies on real legged robots, differences between the simulator and the real system can significantly reduce performance. These differences can come from factors such as actuator dynamics, latency, mass properties, and contact conditions. This problem is commonly known as the sim-to-real gap [Hwa+19; Lee+20]. Hwangbo et al. [Hwa+19] showed that careful simulation and system design can help transfer learned policies to a real robot. This problem can also be addressed with domain randomization [Tob+17]. During training, physical parameters are varied across simulation environments. The policy, therefore, encounters a range of dynamics rather than a single fixed model. The goal is to make the real robot behave like another variation within the training distribution. This can improve the robustness of the learned policy and facilitate sim-to-real transfer [Tob+17; Pen+18b; Tan+18].

To achieve agile locomotion on complex terrain, modern learning-based methods commonly separate locomotion control from online adaptation [Kum+21; Aga+22; Don+26]. Rapid Motor Adaptation (RMA) [Kum+21] introduced a two-stage teacher-student architecture. First, a teacher policy is trained with RL in simulation. The teacher has access to privileged information about the robot and its environment. This can include factors such as terrain properties, payload mass, and contact information. The teacher learns how to use this information to adapt its behavior. Next, a student adaptation module is trained using supervised learning. It receives only a history of proprioceptive observations. From this history, it learns a latent representation of the current environment. The resulting representation is then used by the control policy to adapt its behavior online [Kum+21]. Related approaches also use fast online adaptation to handle changes in the environment and robot dynamics [Mar+22].

The same teacher-student principle has also been applied to locomotion with exteroceptive sensing. Lee et al. [Lee+20] trained a privileged teacher with access to information about the terrain and contact state. The teacher was then distilled into a purely proprioceptive student. The training used online data aggregation inspired by DAgger [RGB11]. Despite lacking exteroceptive sensors, the student learned to handle challenging situations, such as collisions and foot-trapping events [Lee+20]. Related visual approaches retain the privileged teacher but provide the student with camera input. The privileged teacher still had access to a terrain representation, such as a local elevation map. Instead, the student received depth images from an onboard camera. Convolutional networks were then used to extract useful visual features for locomotion [Aga+22; Che+23].

Despite the strong performance of visual PPO-based locomotion policies on challenging obstacle courses [Che+23; Zhu+23], model-free RL remains sample-inefficient. Training can require a very large number of simulated interactions. This is possible in practice because

modern simulators can run thousands of environments in parallel [Rud+22; Mak+21]. The main limitation is that model-free RL treats the simulator as a source of sampled experience. It does not directly use the derivatives of the foundational physics. Policy gradients are therefore estimated from sampled trajectories and their rewards. This can require billions of simulated transitions, even with massive parallelism, which keeps training time relatively short [Rud+22]. Differentiable simulation offers a different approach. Instead of relying only on sampled rewards, it provides gradients through the simulated dynamics. These gradients can contain information about how changes in the state or control inputs affect the final objective [Suh+22; Met+22].

2.3 Analytic Policy Gradients and Differentiable Simulation as a Learning Framework

Differentiable simulation integrates physics-based modeling with the end-to-end learning capabilities of deep reinforcement learning [SKS24; Deg+18; SS24]. The central concept is to render the physics simulation differentiable. This allows gradients of the task loss to propagate through the simulated trajectory and back to the neural network policy parameters [Deg+18; SKS24].

This gradient computation can be viewed as backpropagation through the simulated trajectory. It relies on backpropagation through time (BPTT) and automatic differentiation [Wer90; Bay+18; New+24]. This is different from model-free policy gradient techniques such as REINFORCE and PPO. These methods do not differentiate through the environmental dynamics. Instead, they estimate policy gradients from sampled trajectories and their returns [Rec18; Sch+17b]. In differentiable simulation, the state transition is explicitly differentiated. The simulator is represented as $s_{t+1} = f(s_t, a_t)$, and gradients are propagated through this function [Xu+22; Son+23]. For a trajectory loss L , the gradient with respect to the policy parameters can therefore be computed by applying the chain rule over the simulation horizon [Met+22; SKS24]:

$$\nabla_{\theta} L = \sum_{t=1}^T \frac{\partial L}{\partial s_t} \sum_{j=1}^t \left(\prod_{k=j+1}^t \frac{\partial s_k}{\partial s_{k-1}} \right) \frac{\partial s_j}{\partial a_{j-1}} \frac{\partial a_{j-1}}{\partial \theta} \quad (2.4)$$

Here, $\partial s_k / \partial s_{k-1}$ describes how the simulator state changes with the previous state. The term $\partial s_j / \partial a_{j-1}$ describes the effect of the control input on the state. Finally, $\partial a_{j-1} / \partial \theta$ describes how the policy action changes with its parameters [Suh+22; SKS24]. When the dynamics are smooth, such an approach can provide more direct and stable gradients than its sampling-based counterpart. This can also make training more efficient, reducing the number of simulations [Suh+22; Zha+25].

Differentiable physics engines have faced considerable computational and implementation challenges [New+24]. Early methods often used automatic differentiation (AD) to compute gradients through the simulator. The problem with this approach is that it must store many intermediate states, resulting in high memory usage [Bay+18; New+24; Deg+18]. More recent approaches use GPUs for computations and parallel tensor operations to improve performance. Examples include Brax [Fre+21] and Dojo [How+25]. Other methods compute the gradients directly from the contact solver rather than using automatic differentiation [Wer+21]. This approach can be much faster than finite-difference methods, especially for robots with multiple joints and ground contact [Wer+21; How+25].

Although analytical gradients can be advantageous, they are difficult to use reliably for tasks with frequent contact, such as quadrupedal locomotion [Suh+22; Met+22; Xu+22].

The main problem is that contact with the ground is not smooth. A foot can suddenly touch or leave the ground, and friction adds further constraints and nonlinear effects [Rai+83; New+24; Suh+22]. These effects make the dynamics difficult to differentiate around contact events. The transition Jacobian $\partial s_k / \partial s_{k-1}$ can change rapidly or become poorly conditioned. This can produce gradients that are very small, very large, or unstable. In such scenarios, the gradient may provide limited useful information for training the policy [New+24; Suh+22; Met+22].

This problem becomes more pronounced when transition Jacobians are multiplied over long simulation horizons [Xu+22]. Metz et al. [Met+22] showed that inside chaotic dynamical systems, small changes in the initial state can grow rapidly over time. As a result, gradients calculated through long trajectories can become increasingly noisy or poorly conditioned. Backpropagation over the long horizon can therefore make policy optimization more difficult and may lead to unstable updates or slower learning [Suh+22; Met+22].

Several strategies have been developed to mitigate these gradient-related challenges [New+24]. One is randomized smoothing. It adds noise to the objective and computes the gradient of the resulting smoothed objective. This can make a non-smooth optimization problem easier to handle [New+24]. Suh et al. [Suh+22] study when this type of first-order gradient estimator can be more effective than zeroth-order methods.

Trajectory-space optimization represents another approach, as exemplified by PODS [Mor+21]. This method initially refines a trajectory using gradients from the differentiable simulator. It then uses supervised learning to train the policy to replicate the optimized actions. This process eliminates the necessity of learning a value function concurrently.

A third strategy is the Short-Horizon Actor-Critic (SHAC) formulation [Xu+22]. SHAC restricts backpropagation to short simulation horizons. At the end of each horizon, a learned value function estimates the remaining return. This approach avoids backpropagating through lengthy, potentially unstable trajectories while still incorporating rewards beyond the truncated horizon.

In the context of quadruped locomotion, Song et al. [SKS24] proposed a hybrid approach called α -alignment. Rather than backpropagating through a high-fidelity differentiable simulator, the method uses two simulations in parallel. A full-fidelity, non-differentiable simulator, NVIDIA Isaac Gym, provides the reference states. A simplified analytical SRBD model provides the differentiable surrogate. The surrogate is aligned with the reference state at each step, and its analytical gradients are propagated through a short, truncated horizon [SKS24].

This approach demonstrates sample-efficient learning of blind proprioceptive gaits. The authors also discuss large-scale training and vision-based control as the outstanding demonstrations [SKS24]. Subsequently, Luo et al. [Luo+24] applied first-order policy gradients to the control of perceptive quadruped robots. Their method learns a residual policy on top of a baseline controller using a SHAC-style, critic-bootstrapped horizon. They use a general articulated simulator for obstacle avoidance and vision-based local navigation. Related research has also demonstrated end-to-end visual control via differentiable physics for aerial robots [Zha+25]. In these approaches, however, visual or terrain information is primarily incorporated into the policy through observation input. To author’s knowledge, prior work has not explored routing terrain geometry directly into the analytical loss gradient of an α -alignment-style surrogate while keeping this gradient pathway separate from the observation pathway.

This is the gap the present work addresses. The α -alignment framework is used to study how exteroceptive terrain information interacts with analytical BPTT gradients. A differentiable height-map sampler is incorporated into the SRBD surrogate loss. This allows terrain information to influence the gradient of the policy parameters via the simulator. The proposed approach is evaluated against a baseline in which terrain information is provided solely

as a policy observation.

2.4 Perception for Locomotion

The integration of exteroceptive perception is important for enabling legged robots to traverse complex, unstructured environments at high speed and with high energy efficiency [Mik+22; Lee+20; Don+26]. Without perception, a robot must rely mainly on proprioceptive feedback and reactive control. This approach can work in relatively simple environments, but it becomes more challenging in difficult terrain. In particular, the robot may react too late to obstacles, increasing the likelihood of collisions or foot-trapping events [Mik+22; Lee+20].

Established approaches to robotic perception commonly used a separate mapping and planning pipeline [Aga+22; Fan+18]. Onboard depth sensors or LiDAR were used to estimate the terrain and build an elevation map. These systems could also account for uncertainty in the robot’s localization when updating the map [Mik+22; Fan+18]. The resulting terrain map was then used by a separate planner to select suitable footholds and generate a safe motion plan [Aga+22; Jen+22]. TAMOLS is a representative example of this approach [Jen+22]. It jointly optimizes the robot’s base motion and footholds using a terrain heightmap. However, these structured pipelines can be sensitive to sensor noise and localization errors. Even small errors in the terrain map can lead to poor foothold choices, such as stepping near a gap or hitting an obstacle. [Aga+22; Mik+22].

Deep reinforcement learning has also enabled the direct use of visual or depth data for locomotion control [Aga+22]. These approaches do not require an explicit geometric reconstruction of the terrain. Instead, the policy learns to use terrain information directly from observations [Rec18; Aga+22]. Another approach is to combine proprioceptive and exteroceptive information. Miki et al. [Mik+22] use a recurrent encoder with an attention mechanism to combine both input types. The policy can learn when to rely on exteroceptive information and when to fall back on proprioception. This makes the controller more robust when terrain observations are noisy or unreliable. Across the literature discussed in this subsection, exteroceptive information is provided in two main forms: privileged terrain scans and observations from onboard sensors.

The privileged terrain modality provides direct access to the terrain height around the robot. It is usually represented as a two-dimensional grid of height samples around and below the robot’s feet, often referred to as terrain scan points or “scandots” [Aga+22; Che+23]. In simulation, these values can be obtained directly from the ground truth terrain geometry. This gives the policy information that would not be available from onboard sensors alone. For this reason, privileged terrain information is used during simulation rather than as a deployable input [Lee+20].

In contrast, onboard sensors provide deployable exteroceptive observations, such as depth images or point clouds. These measurements are affected by occlusions, sensor noise, and the sensor’s limited field of view [Don+26; Aga+22]. A common way to connect the two modalities is teacher-student learning. A teacher policy is trained using privileged terrain information, while a student policy learns to reproduce its behavior using only deployable sensor observations [Che+23; Lee+20]. Similar end-to-end approaches have also been used for high-speed visual control in aerial robots [Loq+21; Zha+25].

Simulating realistic visual sensors could considerably reduce the throughput of massively parallel simulations [Aga+22]. This becomes especially important when training with thousands of parallel environments. Standard rendering methods add additional computational cost and can limit the simulation speed [Don+26; Rud+22].

Recent approaches address this problem with lightweight camera simulation [Don+26]. For example, the MGDGP model [Don+26] uses a camera module implemented in NVIDIA Warp [NVI26]. Instead of relying on a full rendering pipeline, it uses GPU-based ray casting to obtain depth observations from the simulator’s collision geometry. This reduces the cost of generating depth images and makes large-scale end-to-end visual training more practical [Don+26].

This thesis brings these perceptive approaches together in a differentiable simulation framework. It follows the latter architecture described above. A privileged, yaw-aligned local height scan of 187 points serves as the reference for the maximum terrain information available to the policy. A lightweight depth camera, based on the Warp ray-casting approach of MGDGP [Don+26], provides deployable visual input that a CNN-based policy learns to use. The two are deliberately not reported as a head-to-head comparison. The height scan is noiseless and complete, and it includes terrain the robot cannot see, so the privileged variant is guaranteed to be superior. The test aims to check instead how big is the blind-to-privileged gap that the deployable sensor recovers. And, where it recovers little, which structural property of the control formulation prevents it.

The mechanism at the center of this is the bilinear height map sampler, implemented as a differentiable operator. This allows terrain gradients from the loss to propagate through the analytical SRBD model and into the policy network. The goal is to test whether this direct gradient-based coupling between terrain information and the policy improves rough-terrain locomotion compared with supplying the same terrain signal only as a policy observation.

Chapter 3

Method - Baseline, Terrain Perception and Structural Limits

3.1 The Differentiable SRBD State Alignment Method

The core training loop in this project uses a differentiable Single Rigid Body Dynamics surrogate model coupled with a high-fidelity physics simulator (NVIDIA Isaac Gym). Since legged robots are complex systems that experience sudden contacts, discrete friction changes, and need to adhere to their joint limits [Rai86], differentiation over longer horizons becomes problematic [Suh+22; Met+22]. This project’s state alignment method based on the approach of Song et al. [SKS24] addresses this problem. Isaac Gym computes accurate, non-differentiable physical states, while the analytic SRBD surrogate provides stable first-order gradients. To prevent gradient explosion or decay, each state alignment happens over a short unroll window.

3.1.1 Centroidal Dynamics of the SRBD Surrogate

The Unitree Go2 [Uni26a] quadruped robot is modeled as a single rigid body. Reducing such robots to their base only is a common method used, for example, on the MIT Cheetah or ANYmal platforms [Di +18; Win+18]. In the code, the body carries a constant mass $m = 7.0\text{kg}$. The diagonal inertia tensor corresponds to $I = \text{diag}(0.024, 0.098, 0.107) \text{ kg} \cdot \text{m}^2$ in its body-fixed frame. Both come straight from the trunk link of the Go2 URDF (6.921 kg, with $I_{xx} = 0.0245$, $I_{yy} = 0.0981$, $I_{zz} = 0.107$). In the SRBD model, legs don’t have any mass and carry no inertia, although they transmit the forces. The forward rollout from the surrogate model is aligned with the ground truth every step. This results in missing mass not accumulating into the trajectory. It only rescales the gradient that the surrogate hands back. The robot interacts with the ground only through four-point contact forces at its feet ($f_i \in \mathbb{R}^3 \mid i \in \{\text{FL}, \text{FR}, \text{RL}, \text{RR}\}$).

The surrogate state at timestep t corresponds to $x_t = [p_t, q_t, v_t, \omega_t]^T$. Where $p_t \in \mathbb{R}^3$ is the center of mass (CoM) position in the world frame W . $v_t \in \mathbb{R}^3$ being the CoM linear velocity in W . $q_t \in \mathbb{H}$ is the unit quaternion representing body orientation relative to W in the $wxyz$ convention. Finally, $\omega_t \in \mathbb{R}^3$ is the body angular velocity expressed in the local body frame B to match Euler’s rotational equations.

The translational dynamics equations are integrated with a time step of $dt = 2 \text{ ms}$ (500 Hz). The position advances at the old velocity, and the velocity is updated afterward.

$$\ddot{p}t = \frac{1}{m} \sum_i \mathbf{f}_i + g \quad (3.1)$$

$$p_{t+1} = p_t + v_t \cdot dt \quad (3.2)$$

$$v_{t+1} = v_t + \ddot{p}_t \cdot dt \quad (3.3)$$

Here $g = [0, 0, -9.81]^T$ m/s² is the world frame gravity vector. The ground reaction forces f_i are also expressed in the world frame.

SRBD surrogate predicts the world frame position of the foot from its current pose and 2D x and z axis kinematics driven by the reference twelve target joint angles that the robot is told to hold, as specified by the policy. This is one of the two paths through which the policy reaches the dynamics. The torque arm $r_i \in \mathbb{R}^3$ from the CoM is:

$$r_i = p_{\text{foot},i} - p_t \quad (3.4)$$

The net external torque in the world frame representation ($\tau_{\text{world}} \in \mathbb{R}^3$) is accumulated and acts on the base:

$$\tau_{\text{world}} = \sum_{i=1}^4 r_i \times f_i \quad (3.5)$$

It is then rotated into the local body frame with the help of the rotation matrix $R(q_t) \in \text{SO}(3)$ derived from the quaternion q_t :

$$\tau_{\text{body}} = R(q_t)^T \tau_{\text{world}} \quad (3.6)$$

Then the local angular acceleration ($\dot{\omega}_t \in \mathbb{R}^3$) is integrated:

$$\dot{\omega}_t = I^{-1} (\tau_{\text{body}} - \omega_t \times (I \omega_t)) \quad (3.7)$$

$$\omega_{t+1} = \omega_t + \dot{\omega}_t \cdot dt \quad (3.8)$$

The rotational part of the integrator is semi-implicit. The new angular velocity (ω_{t+1}) is computed first. Then the orientation is updated with that value. The quaternion rate is half the quaternion product of the orientation with the angular velocity:

$$\dot{q}_t = \frac{1}{2} q_t \otimes \begin{bmatrix} 0 \\ \omega_{t+1} \end{bmatrix} = \frac{1}{2} \Omega(\omega_{t+1}) q_t \quad (3.9)$$

where $\Omega(\omega)$ is the skew-symmetric quaternion multiplication matrix:

$$\Omega(\omega) = \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix} \quad (3.10)$$

Finally, the quaternion state is updated and normalized:

$$q_{t+1} = \frac{q_t + \dot{q}_t \cdot dt}{|q_t + \dot{q}_t \cdot dt|} \quad (3.11)$$

These steps are the whole run of a single analytical surrogate iteration.

3.1.2 Mapping Policy Outputs to Ground Reaction Forces

The policy network does not output the forces directly. It returns 12 joint-angle residuals instead. These are then squashed using a $\tanh$ activation and scaled per joint type (0.10 rad for hips, 0.30 rad for thighs and calves). Finally, they are added to the default standing posture ($q_{\text{stand}} = [0.0, 0.86, -1.40]^T$ rad per leg). This yields the reference joint position vector $q_{\text{ref}} \in \mathbb{R}^{12}$. Then the PD controller drives the robot's joints toward the desired angles q_{ref} . The measured joint positions and velocities are detached so that gradients flow only through the desired joint angles (q_{ref}).

The joint torques become ground forces through the foot Jacobians. $J_i(q) \in \mathbb{R}^{3 \times 3}$ relates foot to joint velocities. The Jacobians are obtained from Isaac Gym in the world frame and are combined into a block-diagonal matrix J . The contribution of each leg is weighted by the stance mask $s_i \in \{0, 1\}$. This indicates whether the leg is in contact with the ground. The relation between joint torques and foot forces can then be written as $\tau = -J^T y$. When some legs are in the swing phase, J becomes rank deficient because those legs are not contributing ground contact forces. To make the force estimation numerically stable, a damped pseudo-inverse with $\sigma_{\text{damp}}^2 = 10^{-9}$ is being used:

$$J = \text{diag}(s_1 J_1, s_2 J_2, s_3 J_3, s_4 J_4), \quad (JJ^T + \sigma_{\text{damp}}^2 I) y = J \tau, \quad f = -y \quad (3.12)$$

The force estimation is stabilized using Singular Value Decomposition. The singular values below $\sigma_{\text{floor}} = 10^{-3}$ are replaced by the floor value. Swing legs are assigned zero contact force. The estimated forces are then constrained by limiting the vertical component to $[20, 250]$ N and enforcing the friction cone condition $|f_{i,xy}| \leq 0.6 f_{i,z}$. The Jacobian is treated as a non-differentiable mapping, so gradients reach the contact forces through the joint torques τ .

3.1.3 The State Alignment Coupling Formulation

To prevent the surrogate model from drifting away from the true state, SRBD values are coupled with Isaac Gym's values at every timestep. With $x_{\text{gt},t} \in \mathbb{R}^{13}$ the ground-truth state (base position, quaternion, world linear velocity, body angular velocity), $x_{\text{srbd},t}$ the surrogate's prediction, and $\alpha = 0.9$ set as standard:

$$x_t \leftarrow x_{\text{gt},t} + \alpha \cdot (x_{\text{srbd},t} - \text{detach}(x_{\text{srbd},t})) \quad \implies \quad \frac{\partial x_t}{\partial \theta} = \alpha \cdot \frac{\partial x_{\text{srbd},t}}{\partial \theta} \quad (3.13)$$

In the forward pass, the bracket evaluates to zero, $x_t = x_{\text{gt},t}$. The surrogate is re-pinned every step, and the modeling error cannot compound. Backward $x_{\text{gt},t}$ is a constant, and detach stops only one gradient flow. This allows combining the accuracy of a high-fidelity simulation with a smooth gradient from the simplified model.

A truncated backpropagation through time over a horizon of $T_{\text{rollout}} = 24$ simulation steps (48 ms) is being used. The policy is queried every 5 physics steps, resulting in 5 control decisions per rollout. After each rollout, the accumulated loss is backpropagated, and the policy is updated using AdamW [LH19]. The simulation state is then detached before the next rollout starts. The short horizon helps maintain stable gradients while keeping the training efficient [SKS24; Xu+22].

3.2 The Forked Implementation

The project began as a fork of a third-party implementation of this method. The upstream repository authored by Ronggan Zhang between 1 February and 21 April 2026 carries no license file, so all rights remain reserved by default [Zha26]. It was given as a starting point for this thesis. It was forked at the commit `'17314532585369d58f6d6d0111ebcb8b96536b9f'`. Upstream supplied the batched PyTorch SRBD integration, the phase-driven gait scheduler, the joint PD control scheme, the flat ground velocity-tracking losses, and the α -alignment training loop.

Three further projects were vendored, adapted, or used as conceptual inspiration, each attributed at its point of use:

legged_gym (Rudin et al. [Rud+22])

The terrain generator implemented in `terrain.py` (`Terrain`, `gap_terrain`, `pit_terrain`) was ported with minimal modifications to preserve the original implementation. The spawn-origin assignment, curriculum-based terrain promotion and demotion, and the 187-point height scan were adapted to this project. These modifications were made in accordance with the BSD-3-Clause license.

MGDP (Dong et al. [Don+26])

The depth-range ray-casting kernel from the `warp_sensor` module (`cam_kernel.py`) was integrated into this project's pinhole camera implementation. Only the depth-range sensing functionality was retained, as the LiDAR, point cloud, segmentation, and surface-normal kernels were not required for the proposed simulation setup. The `warp_sensor` package declares itself MIT-licensed (`pyproject.toml`), and that license applies to the used code.

DiffPhysDrone (Zhang et al. [Zha+25])

This repository and the paper had only a conceptual influence. No code used. The depth CNN training using differentiable rollouts and the 12×16 depth resolution was inspired by this paper's ideas and findings.

3.3 Differentiable Terrain Height Sampling

To prevent noisy gradients, a continuous terrain is required for the simulation. This project uses a differentiable bilinear sampler for the terrain heightfield, aligned with the robot's yaw orientation. It can provide a policy with two fields. The exact grid, which serves as a 187-point height-scan observation and a Gaussian-blurred copy ($\sigma = 2.0$ cells), is used in the loss path.

The sampled terrain height is being made differentiable with respect to the horizontal foot position. The terrain height is sampled at the predicted foot positions, calculated by the SRBD model. This allows the local terrain slope ($\partial z / \partial xy$) to be propagated back through the differentiable surrogate (SRBD) to the policy:

$$z_{\text{terrain}} = \text{grid_sample}(H_{\text{blurred}}, p_{xy}) \quad (3.14)$$

The sampled terrain heights are used in two parts of the loss. Firstly, as a reference for the base height, and secondly, to ensure sufficient foot clearance during the swing phase. The

required clearance (c_i) varies over the swing phase (s_i). It reaches its maximum at mid-swing and decreases to zero at the beginning and end of the swing. This allows the foot to approach the terrain at touchdown while still encouraging sufficient clearance during the swing:

$$L_{\text{clear}} = \left\langle \left(m_i^{\text{sw}} \max(0, z_{\text{terrain},i} + c_i - z_{\text{foot},i}) \right)^2 \right\rangle, \quad c_i = 4h s_i(1 - s_i), \quad s_i \in [0, 1] \quad (3.15)$$

Here h is the swing-height target, m_i^{sw} the swing mask, and $\langle \cdot \rangle$ the mean over feet and rollout steps.

Blurring the heightfield smooths sharp terrain edges and provides non-zero gradients over a larger area around obstacles. This should theoretically allow the gradient to provide useful information even when the queried foot position is not directly on the terrain boundary.

3.4 Depth Camera Vision Policy

The MLP policy was extended to include a depth camera input to test exteroceptive sensing without providing the policy with an explicit terrain map. The depth camera is simulated using GPU ray casting in NVIDIA Warp. A total of 48×64 rays are launched against the terrain mesh. Then the resulting depth measurements are resized to a 12×16 depth image. A small convolutional neural network encodes this image into a 64-dimensional feature vector. The resulting feature vector is then provided to the policy as a representation of the observed terrain.

The depth encoder consists of two convolutions with a stride of 2, followed by a linear layer. It contains approximately 30k parameters. The resulting feature vector is concatenated with the 36-dimensional proprioceptive observation. This yields a 100-dimensional input to the same 256×256 policy network used by the blind policy.

The depth image itself is treated as a non-differentiable input. The encoder weights, however, remain differentiable. Therefore, gradients from the loss can update the encoder through the predicted joint targets. During training, the loss has access to the ground-truth terrain height, whereas the policy receives only depth observations and proprioceptive information.

$$\frac{\partial L}{\partial W_{\text{enc}}} = \frac{\partial L}{\partial x_{\text{srbd}}} \frac{\partial x_{\text{srbd}}}{\partial q_{\text{ref}}} \frac{\partial q_{\text{ref}}}{\partial \mathbf{z}_t} \frac{\partial \mathbf{z}_t}{\partial W_{\text{enc}}} \quad (3.16)$$

This is how the gradient path can be expressed. L denotes the loss function, W_{enc} represents the weights of the depth encoder, and $\mathbf{z}_t$ is the feature vector produced by the encoder from the depth image. The vector q_{ref} contains the desired joint angles generated by the policy, while x_{srbd} represents the robot state predicted by the SRBD model. The idea was taken from the work of Zhang et al. [Zha+25].

3.5 Loss Design

The training objective is defined as a weighted sum of eight loss terms. Seven are inherited from the baseline implementation [Zha26]. The eighth term penalizes insufficient foot clearance above the terrain:

$$L = a_1 L_v + a_2 L_h + a_3 L_\omega + a_4 L_{\text{ctrl}} + a_5 L_{\text{proj}} + a_6 L_{\text{foot}} + a_7 L_{\text{yaw}} + a_8 L_{\text{clear}} \quad (3.17)$$

Linear velocity ($a_1 = 10$)

Squared error between the body frame v_{xy} and the command. Yaw is not in this term.

Base height ($a_2 = 1.0$)

Absolute deviation of the base height from $h_0 = 0.35$ m in relation to the terrain.

Angular rate ($a_3 = 0.01$)

Squared roll and pitch rate plus squared yaw-rate tracking error.

Control effort ($a_4 = 0.01$)

Squared norm of the twelve joint angle offsets produced by the policy.

Attitude ($a_5 = 0.5$)

Squared norm of the horizontal gravity projection, $\|g_{xy}/g\|^2$. Penalizes deviations from an upright body orientation.

Swing-foot tracking ($a_6 = 5.0$)

Squared deviation of the swing foot from the planner's reference p_{ref} .

Heading ($a_7 = 0.1$)

Squared yaw error between the actual and commanded yaw angles.

Foot clearance ($a_8 = 3.0$)

The terrain term penalizes insufficient clearance between the swing feet and the terrain. It always remains in the sum, but evaluates to exactly zero unless the run enables the terrain loss path.

Which terms are live depends on the run. The blind reproduction on the flat ground uses only terms 1-7. In such a run, the height datum is in relation to the world frame (inherited). In the `legged_gym` curriculum, it becomes terrain-relative, including for the proprioceptive policy. The clearance term switches only if the policy receives terrain information as a gradient, either from a depth camera or from a Gaussian-blurred height scan.

The policy is trained over 24 simulation steps (48 ms) at a time. The AdamW optimizer is used with a learning rate of 10^{-3} . To keep the training stable, the global gradient norm is limited to 0.3.

3.6 Structural Limits

Several conditions must be met for the terrain signal to change the policy's behavior. During this project, three factors were identified that might impact how this architecture responds to the additional terrain information. Firstly, the policy needs an output capable of addressing such an impact. Secondly, the differentiated horizon must be long enough for that output to have a consequence. At last, the objective needs to somehow encode the failure caused by rough terrain and improper handling. This architecture appears to satisfy none of the three.

3.6.1 Limit 1: No Perceptual Output Channel

The policy’s output is 12 joint-angle residuals. The L_{foot} with a weight of 5 (second largest) makes the swing foot follow a reference computed from the velocity command, base pose, and gait phase alone. These reference positions do not depend on the terrain. This means that the objective contains a term that actively opposes any correction coming from the terrain information. For example, if there is a step in front of the robot, the policy might use the observation to raise the foot, but L_{foot} still penalizes deviations from the original reference trajectory. Moreover, this reference trajectory assumes a flat terrain height at touchdown. The target foot height is based on the height of the last step. This can be incorrect in hard terrain. Although Song et al. [SKS24] have shown that their policy was successful on easy terrain.

3.6.2 Limit 2: Truncated Gradient Horizon

The differentiable rollout is only 24 simulated steps, corresponding to 48 ms. At a gait frequency of 1.6 Hz, one cycle takes 625 ms. So the differentiated windows are only a small part of this cycle. No complete event can enter it. The optimizer will not see the consequences of certain foot placement.

3.6.3 Limit 3: Termination Outside of the Gradient

An episode is terminated if the roll or pitch angle exceeds 51.6° . This termination does not provide a differentiable penalty for tipping over. Thus, the loss does not directly say that the robot is getting close to falling. The L_ω penalizes roll and pitch rates rather than the angles themselves. The L_{gproj} is related to body orientation. As a result, there is no strong loss term that increases as the robot approaches the termination threshold.

These limitations can compound. Foot placement based on terrain information is difficult if reference-tracking loss discourages deviations from the predefined foot trajectory, and the termination condition does not provide a direct gradient to avoid large body tilts. This means that the terrain perception might not be enough.

3.7 Interventions on the Structural Limits

To assess whether the aforementioned limits could be bypassed, four targeted interventions were developed.

3.7.1 Foothold Correction Channel

The terrain-aware foothold correction mechanism was implemented in two steps, as two separate interventions. Firstly, landing height and swing apex are sampled from the differentiable height field rather than the height of the last contact. Secondly, the policy output is extended to 16 values. Those additional values provide one sagittal foothold correction for each leg. Maximum displacement of the correction is $\pm 0.10\text{m}$. A lateral correction was

also implemented (increasing output to 20 values), but it is disabled by default. This stems from the SRBD model not representing the corresponding lateral leg degree of freedom. The foothold correction is detached when computing the swing-foot tracking loss L_{foot} . This prevents a degenerate scenario in which the reference position is dragged to the current foot position instead of moving it to the desired one. To make the policy choose a stable foothold, a new foothold-quality loss was introduced. For each planned foothold, the terrain heights are sampled around the target point in a ring pattern. Then they are compared with the height at the center:

$$L_{\text{fq}} = \left\langle m_i^{\text{sw}} \cdot \frac{1}{k} \sum_{j=1}^k (z_{\text{ring},ij} - z_{\text{centre},i})^2 \right\rangle \quad (3.18)$$

A flat terrain surface produces a value close to zero. Terrain edges or holes yield higher values due to height differences around the foothold. The loss is evaluated on the blurred terrain scan so that the resulting terrain signal also provides useful gradients. There is also another term (L_{fres}), that forces the policy to use the corrections only when necessary. Every non-zero correction carries a quadratic cost that shrinks it back to the uncorrected foothold unless L_{fq} rewards the displacement by more than it costs. The weight of the foothold quality term (L_{fq}) was selected to contribute approximately 5% to the total loss. The weight on the foothold correction L_{fres} was chosen to contribute 2% to the total loss.

3.7.2 Extended Gradient Horizon

A probe with an extended differentiable rollout to 48 steps was also run, corresponding to the 96 ms window.

3.7.3 Differentiable Termination Surrogate

To address the third limit, an additional loss term was introduced. It penalizes excessive body tilt. It is based on the gravity projection already used in the loss function:

$$L_{\text{tilt}} = \left\langle \max(0, \cos \theta_{\text{on}} - \cos \theta)^2 \right\rangle \quad (3.19)$$

The penalty activates when the body tilt exceeds $\theta_{\text{on}} = 0.6$ rad (34°). This threshold is lower than the original one used for termination. It gives the policy a gradient signal before the robot reaches the termination condition. During normal walking, the trained policy tilts up to around 16° . Then the penalty remains inactive. The barrier is evaluated using combined body tilt. The original termination condition uses roll and pitch separately. Two different weights for this loss term were tested. One is 5% and the second is 15% of the total objective. The activation frequency was also recorded. This allows to distinguish between a term that has no effect and a term that was simply not activated during the experiment.

Chapter 4

Implementation - Vectorization and GPU Acceleration

4.1 Software Architecture and Design Patterns

To answer the first research question of this thesis and to run experiments for the two remaining research questions, the project's limiting factors needed to be mapped and its simulation throughput improved. Otherwise, the experiments would take several months, instead of one day of wall-clock time for the simulations.

The inherited codebase was already structured into modules. Each module isolates physical abstractions, gait planning, neural network learning, the main simulation loop, and mathematical operations. A challenge in coupling such a simulation framework is the possibility of circular dependencies and high dispatch overheads. These were already resolved with two core design patterns.

The Attribute Proxy Mixin Pattern

The state of the environment, gait phase, and other parameters are used simultaneously by three modules. The physical simulator (`env.py`), the gait controller (`gait.py`), and the SRBD model (`srbd.py`). To prevent copying the same data between modules, the gait planner and the SRBD model keep a reference to the main environment and access its attributes directly. It is done by overriding Python's `__getattr__` and `__setattr__` methods. For example, when the SRBD model writes one of its variables (ex. `self.srbd_p`), the value is stored directly in the environment's state. This enables the sharing and modification of the same physical variables without circular references and unnecessary CPU-GPU copies.

Eager Tensor Binding and Configuration Driven Mode Selection

Global settings, physical parameters, and experimental options are defined in the configuration file (`config.py`). The required PyTorch tensors are allocated when the environment is created. Isaac Gym's DOF state tensors, root state, SRBD state, contact forces, and Jacobians are all wrapped using `gymtorch.wrap_tensor`. At the startup, the persistent buffers used by the robot are also allocated. Similarly, the depth camera data is connected to NVIDIA Warp without creating additional copies. During rollout, the temporary values are stored in Python lists. These are then stacked into tensors once per iteration, keeping data needed for BPTT local to each rollout. The simulation is controlled from Python. The computationally intensive operations run on a GPU (either through PyTorch or custom CUDA kernels). This allows the CPU-side logic to run while GPU computations are in progress.

4.2 Pipeline Vectorization and Batching

The inherited codebase was limited by sequential per-robot Python loops used for simulation rollouts. They performed coordinate transformations, calculated the foot positions using forward kinematics, converted joint torques into foot forces for each environment, and updated the gait phase. Since Python is an interpreted, by default single-threaded language, looping over the entire batch created a CPU bottleneck. It resulted in the GPU not being fully utilized. All loops were eliminated, using highly batched PyTorch tensor operations. The environment steps all parallel environments at the same time. This transition required additionally redesigning several key operations:

Batched Coordinate Transforms

All the matrix operations are rewritten as fully batched tensor expressions.

Vectorized Swing Foot Trajectories

The swing foot targets are now computed for all robots and legs at once using quadratic interpolation between the liftoff, apex, and touchdown points. Stance masks are used to identify the legs that are in the swing phase.

Batched Torque Estimate Solver

The foot Jacobians are obtained from Isaac Gym and updated at each simulation step. Then the function responsible for estimating foot forces and respective torques for each joint builds a $(B, 12, 12)$ tensor using the stance masks and solves it for all environments at once. A batched damped pseudo-inverse is used instead of solving a separate 12×12 system for each robot. This computation provides the differentiable connection between the policy's joint targets and the ground reaction forces.

For verification, if the rewrite does not change the physical behavior, the parity tests were created (`tests/test_vectorization.py`). They provide the same randomized input to both the old and new batched implementations and compare the results. For usage of `float32`, the absolute errors of the calculated values for each operation between the two versions stay within 10^{-3} . An additional test verifies if the gradient is still propagated through the batched Singular Value Decomposition to the joint references.

Additionally, both training processes for the same flat-ground scenario over 1000 iterations were compared. Over the first 50 iterations, the velocity tracking losses differed by 1.5×10^{-4} . Over the last 200 iterations, their mean tracked velocities agree to within 2 mm/s. The difference at the beginning of the training is caused by floating-point arithmetic. Since the training uses differentiation, it is sensitive to small numerical differences. However, the overall training behavior remains equivalent.

4.3 Fused CUDA SRBD Kernel and Analytic Adjoint

Batching the main training loop bypasses Python's looping overhead, but executing a sequence of small PyTorch operators introduces GPU kernel launch latency. Each of such operators (element-wise addition, quaternion norms, etc.) needs to be dispatched separately to the GPU's queue. To overcome this limitation, a custom C++/CUDA extension that performs all SRBD computations for a single simulation step in a single GPU kernel was developed.

4.3.1 Fused Forward Kernel Design

The forward computation of the SRBD model is implemented in CUDA and exposed to Python using pybind11 [JRM26]. The kernel uses 256 threads per block, each thread assigned to one robot. Every thread performs around 200 simple floating-point operations. No shared memory, synchronization, or atomics needed. Each variable, except for fixed hip offsets, is loaded at the beginning of the kernel. The offsets are stored in the GPU's constant memory. The calculations are the same as in the PyTorch version of the SRBD model. The updated state x_{t+1} is written back to the GPU's global memory at consecutive locations assigned to each thread. By performing the complete simulation step in a single CUDA kernel, intermediate results remain in registers and constant memory. This reduces memory traffic compared with the original PyTorch implementation.

4.3.2 Analytical Adjoint Backward Pass

PyTorch Autograd engine records the operations performed during the forward pass to compute their gradients during backpropagation. As the simulation takes 24 steps (48 ms), this requires storing many intermediate tensors. To reduce memory usage, a custom backward kernel was implemented. The kernel is wrapped in a `torch.autograd.Function` and stores only the six input tensors instead of all intermediate values. During backpropagation, the kernel first recomputes intermediate values and then propagates the gradients backward through the SRBD model's equations. This reduces memory usage at the cost of additional computation. The gradients of the final position, velocity, orientation, and angular velocity are propagated back to the inputs in a single CUDA kernel launch.

Let $\bar{x}_{t+1}$ denote the gradients of the loss with respect to the robot state at the next time step (in CoM relation):

$$[\bar{p}_{t+1}, \bar{q}_{t+1}, \bar{v}_{t+1}, \bar{\omega}_{t+1}]^T \quad (4.1)$$

For the translational dynamics,

$$p_{t+1} = p_t + v_t dt, \quad v_{t+1} = v_t + \frac{f_{\text{sum}}}{m} dt \quad (4.2)$$

f_{sum} the total ground reaction force acting on the robot

the gradients are propagated backward as:

$$\bar{p}_t = \bar{p}_{t+1}, \quad \bar{v}_t = \bar{v}_{t+1} + dt \bar{p}_{t+1}, \quad \bar{f}_{\text{sum}} = \frac{dt}{m} \bar{v}_{t+1} \quad (4.3)$$

The position gradient is passed directly because the center of mass position cancels out when computing the moment arm:

$$r_i = p_{\text{foot},i} - p_t = R(q_t) \ell_i(q_{\text{ref}}) \quad (4.4)$$

$R(q_t)$ the rotation matrix corresponding to the robot's orientation q_t
 $\ell_i(q_{\text{ref}})$ the position of the i -th foot relative to the robot's center of mass, computed from the reference joint configuration
 $p_{\text{foot},i}$ the position of the i -th foot

The rotational dynamics do not create an additional gradient with respect to p_t . It flows through the robot orientation q_t and the joint reference q_{ref} .

After each integration step, the quaternion is normalized to maintain the unit norm constraint. The gradient is propagated through this normalization, then through the quaternion and angular dynamics. It continues through the torque and foot-force computations and finally reaches the reference joint angles.

To verify the mathematical correctness of both kernels, the testing suite was developed (`tests/test_srbd_kernel.py`). The forward verification checks whether the differences between the computed values of the PyTorch implementation and the CUDA kernel are within 10^{-4} . For the backward kernel, the randomized gradients are used and compared against PyTorch Autograd for all six input tensors. The results agree within 10^{-3} . An additional test verifies the complete backward pass through the `autograd.Function` wrapper.

The comparison is numerical rather than exact. Small differences are expected since the CUDA kernel uses `--use_fast_math` compilation flag. It approximates functions like `sinf`, `cosf`, and `rsqrtf`. It is acceptable since the SRBD state is corrected with Isaac Gym’s state.

4.3.3 Profiling

To locate any further bottlenecks, the application was profiled. It showed that there is no longer a need to write custom accelerated modules, since the biggest bottleneck is still the simulation step in Isaac Gym. All of the results of the profiling are described in the Chapter 5, as part of the results.

4.4 GPU Perception Stack

To extend Research Question 2 and to check if the exteroceptive variant of this framework could be deployed into a real robot, a perception stack was added. One of the aims was to minimize CPU-GPU memory latency.

4.4.1 Ray Casting Depth Camera

The problem with developing a visual sensor in simulators like Isaac Gym is that one must rely on standard graphics rasterization pipelines. Those execute additional calculations such as texture mapping and dynamic lighting. This can reduce the simulation speed. To overcome this, a lightweight depth camera simulator based on NVIDIA Warp was integrated [NVI26]. The ray casting kernel itself is a trimmed copy of the depth-range kernel (`warp_sensor`) from the MGDGP project [Don+26]. This project supports only depth sensing. The kernel was included directly into the codebase of this thesis to avoid any external dependency. The camera wrapper, zero-copy data handling, CUDA graph capture, and post-processing were implemented as part of this work.

The depth camera is mounted on the torso of the Unitree Go2 at position $[0.28, 0, 0.06]$ m in the body frame relative to the base origin. It is directed 28° downward with a 87° horizontal field of view. The module is compiled for CUDA using NVIDIA Warp. It generates depth measurements using cast rays towards the terrain mesh. A Warp mesh is built from the same vertices and triangle arrays that are handed to PhysX as the collision trimesh. This ensures that the depth image represents the actual simulated geometry. Including vertical stair surfaces that cannot be captured by the heightfield representation.

To avoid unnecessary data transfers between the GPU and CPU, the camera module uses Warp views of existing PyTorch CUDA tensors. Camera pose and pixel data are shared between PyTorch and Warp.

Camera kernel operations are captured in a CUDA Graph. In the subsequent steps, the same operations are replayed using new camera data. This helps to reduce the overhead of launching the kernels.

Firstly, a 48×64 depth image is being generated and then reduced to a 12×16 . Then the image is processed by the CNN, consisting of two stride-2 convolutional layers followed by a linear layer. The encoder produces a 64-dimensional vector. This is then used as input to the policy.

All camera and encoder computations remain on the GPU. At $B = 2048$ over 5000 iterations, the depth-based policy required 2.03 s per iteration compared with 1.55 s for the blind baseline.

4.4.2 The Empirical Reproducibility Floor

Before evaluating the proposed method, the numerical variability of the simulations was measured. Although the SRBD simulator is deterministic, the PhysX solver used in Isaac Gym performs contact resolution in parallel on the GPU. Different execution orders can produce small differences between runs. It was measured with six runs with the same settings. Even at the first iteration, the total loss ranged from 1.740280 to 1.741074. A relative spread of:

$$\frac{\max - \min}{\text{mean}} \approx 4.6 \times 10^{-4}$$

Therefore, approximately 5×10^{-4} is used as the empirical reproducibility floor at initialization. This divergence increases during training. For example, two identical runs differed in one of the loss terms up to 1.4×10^{-5} at iteration 0 to 7.7×10^{-3} at iteration 29. This illustrates how small numerical differences can grow in a differentiable simulation. The effect also varies between the batch size and terrain type. Therefore, the experiments in Chapter 5 are compared against the reproducibility floor measured for the corresponding experimental regime rather than against a single global tolerance.

Chapter 5

Experiments and Results

5.1 Experimental Setup and Statistical Criteria

All experiments were run on a single laptop with an NVIDIA GeForce GTX 1660 Ti GPU (6 GB VRAM), an Intel Core i7-10750H processor, PyTorch 1.13.1, and CUDA 11.7. The computer used Linux Ubuntu LTS 20.04 as an operating system.

5.1.1 Robot and Simulator Parameters

The Unitree Go2 robot model is taken from its official URDF file [Uni26b]. The robot is simulated through the Isaac Gym GPU pipeline at 500 Hz. The policy runs at 100 Hz, deciding the joint position targets every 5 simulation steps. The robot uses a fixed-frequency trot at a step frequency of 1.6 Hz, corresponding to a 625 ms gait cycle.

A base policy receives 36D proprioceptive observation. Three velocity commands (v_x , v_y , turn rate), eight gait phase variables (sine and cosine for each leg), three linear velocities of the body frame, four quaternion components describing orientation of the base, three angular velocities of the body, twelve joint angles (deviation from the standing posture), and three gravity projection variables.

Exteroceptive runs extend the policy's input. The heightscan mode appends a 187-point elevation grid around the robot (17×11 at 0.1 m spacing), resulting in 223D input. The depth camera mode, after compressing the image into a 64-value vector, concatenates the proprioceptive run input with this vector, resulting in a 100D input to the policy. The gradient mode keeps a 36D input, and terrain enters through the differentiable loss. There is also a final mode, a controlled experiment, which uses both the terrain gradient and the height scan as observations.

5.1.2 The Procedural Terrain Curriculum

The terrain curriculum of `legged_gym` [Rud+22] was ported. The environment is a 10×20 grid of $8 \text{ m} \times 8 \text{ m}$ cells. The rows represent ten difficulty levels, and the columns are divided into the following terrain families: smooth slope (continuous ramps), rough slope (ramps with high-frequency height variations), stairs down, stairs up, and discrete obstacles (flat terrain with vertical blocks). The curriculum promotes or demotes each robot in every episode. Episode time-outs at 20 seconds. If, by the end of the episode, the robot covers more than 4 m from its cell origin, it is promoted to the next difficulty level. If it covers less than $0.5 \times \text{episode length} \times \text{commanded speed forward}$, it is demoted. If the robot tips over during the episode, the episode ends early, and the robot is demoted.

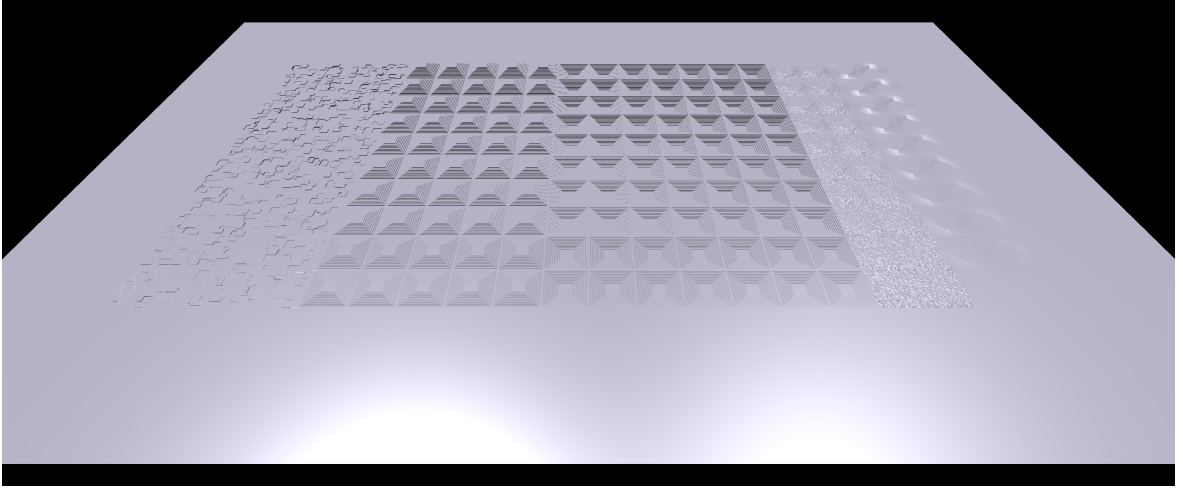


Figure 5.1: Shape of the terrain used in the curriculum.

5.1.3 Statistical Criteria

Three criteria determine how the results are interpreted.

Simulator variability

Isaac Gym simulations are not bit-reproducible. Identical configuration runs, run 46 minutes apart on the same machine, differ by a $\sim 5 \times 10^{-4}$ relative error in the zeroth-iteration loss. Cross-run differences smaller than this value are treated as numerical noise.

Mean terrain level noise

When the curriculum reaches its minimum level, the mean terrain level becomes unstable and fails to clearly reflect the policy’s progress. A few environments that reach a higher level on the smooth slope can increase the mean even when most environments remain stuck at the minimum level. This can create a misleading impression of improvement. These results show this effect. Two runs with the same configuration and seed produced mean terrain levels of 0.043 and 0.025 at the same iteration, a difference of about 70%. Therefore, the maximum terrain level reached within each terrain family is used as the primary metric, with the maximum level on upward stairs being the main indicator. The mean terrain level is reported only as a secondary metric, averaged over the last 100-500 iterations.

Training length consistency

All comparison groups are matched by iteration count. The iteration for each experiment was chosen based on smoke runs to highlight the most important caveats while minimizing simulation time.

5.2 Baseline Reproduction and Parity

To check the parity of the enhanced framework against the inherited baseline, three training runs were conducted with a fixed 0.5 m/s forward command at $B = 16$ robots and 1000

Metric	Inherited reference	Batched PyTorch	Fused CUDA
Initial L_v loss	0.163	0.163	0.163
Final L_v loss	0.0080	0.0056	0.0222
Tail 200-it. mean L_v	0.0134	0.0157	0.0155
Tail 200-it. mean speed v_x [m/s]	0.460	0.455	0.456

Table 5.1: Convergence and velocity tracking parity on flat ground.

iterations. The training was on flat ground, since this is the only mode the inherited implementation supports.

Small numerical differences can accumulate during differentiable simulation, which explains why the values differ slightly. This is expected behavior and does not indicate a change in the underlying physics. Especially for CUDA variant, since it uses `-use_fast_math` flag. The results show that these optimizations preserve the physical model and the optimization process. For the inherited code there is no total loss metric available, so the velocity tracking loss was chosen, since it is the most important factor in the total loss. On the flat ground both PyTorch batched and CUDA kernel versions walk with a 0.00% fall rate.

All three runs show a pronounced transient around iteration 20, during which the mean body tilt increases to 46.8° . At batch size $B = 16$, the policy converges to a stable 3.5° by iteration 100. During training of larger batches, recovery is slower (21.5° at $B = 1024$ and 13.2° at $B = 4096$ at iteration 100). Only the smallest batch converges this quickly, above $B = 16$ the tilt at iteration 100 is much larger, and within the larger batches recovery becomes faster as B grows. This is a property of the method’s early training. However, this behavior is problematic when Rudin’s curriculum is enabled. Because falling is not directly penalized in the loss, the early tilt can cause the curriculum to reduce the terrain difficulty before the policy can stabilize.

5.3 RQ1 - Training Throughput Results

5.3.1 Speed-up Compared to the Baseline

To check the results of the enhancements in the code, three tests were ran. In batch $B = 16$, each variant was proprioceptive on flat terrain and ran 1000 iterations with a fixed 0.5 m/s forward command. Only the small-batch training was used, since the simulation of the inherited codebase takes significantly more time in more parallel environments. Three runs of each variant were ran and the mean total time was taken. The table below describes the time it took per iteration for each variant.

Systems variant	Seconds / iteration	Iterations / second	Speedup vs. inherited
Inherited	4.64	0.216	—
Batched PyTorch	0.488	2.048	9.48×
Fused CUDA	0.387	2.581	11.94×

Table 5.2: Systems throughput decomposition at $B = 16$ on flat ground.

Vectorizing the Python loops delivers the best performance gain.

5.3.2 Throughput Scaling

The variant with the custom CUDA kernel has shown an advantage over the PyTorch-only version in a small number of environments. As the number of environments grows, the advantage is becoming less noticeable.

Batch size B	CUDA [it/s]	PyTorch [it/s]	Speedup	Allocator VRAM (CUDA/Torch)
16	2.581	2.048	1.26×	3.2 MB / 3.3 MB
64	2.421	1.965	1.23×	5.1 MB / 5.3 MB
256	1.934	1.629	1.19×	14.1 MB / 15.0 MB
1024	1.488	1.359	1.09×	53.2 MB / 57.0 MB
2048	1.122	1.070	1.05×	111.0 MB / 118.6 MB
4096	0.731	0.710	1.03×	214.1 MB / 229.3 MB

Table 5.3: Systems throughput over growing batch size. Flat ground, blind policy, fixed forward 0.5 m/s command.

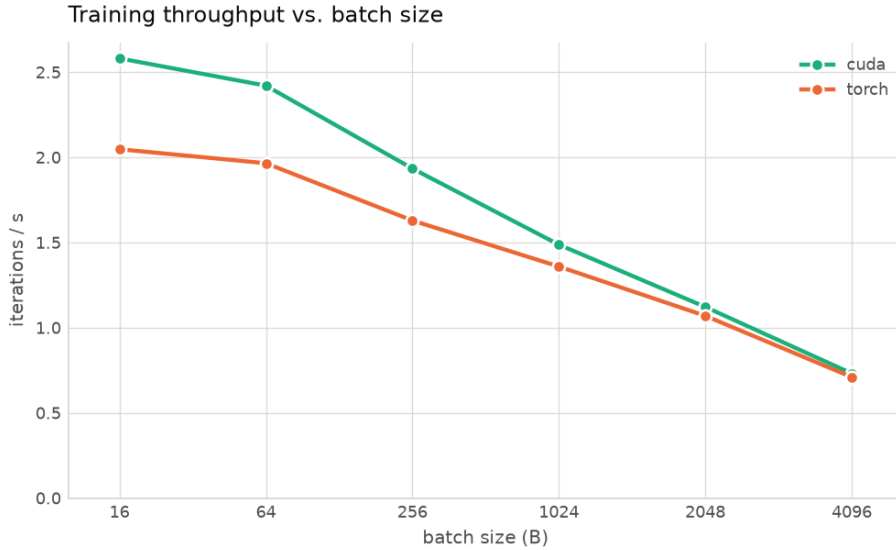


Figure 5.2: System throughput over growing batch size as chart.

All twelve runs were executed in a single session, one run per variant. The configuration for the test runs is the same: seed 0, blind mode on flat terrain, 100 iterations, a 24-step differentiation window. Each figure is the median over the 80 timed iterations of a 100-iteration run, with the first 20 discarded as warm-up because they incur CUDA context creation, cuBLAS autotuning, and allocator growth. In the table the "Allocator VRAM" column shows the peak memory used for live PyTorch tensors. It excludes the CUDA context, Isaac Gym's PhysX buffers and the terrain mesh. The fused kernel's analytic adjoint replaces the stored forward intermediates the autograd tape would otherwise retain.

As the batch size increases, the shrinking of the kernels' relative advantage inverts the usual assumption about using kernels in highly parallel environments. To understand the cause, the profiling of the code was necessary.

For profiling, the py-spy [Fre26] tool was used. It has been shown that after the introduction of a CUDA kernel, the time required for the training step that uses the SRBD model reduced from 5.6% of the total training time to 0.36%. The problem is that most of the time is spent on the simulation running in Isaac Gym. And as the batch grows, the simulation takes

an even larger share of the execution time. As the whole run of the training was profiled, also the startup of the environment was an important part of the whole runtime, but since in the table measurements are taken without the first 20 iterations, it is not considered here.

5.3.3 The Core Performance Takeaway

The answer to what determines the throughput in a differentiable simulation locomotion pipeline is in this scenario the NVIDIA Isaac Gym simulator. A custom-developed accelerator, like CUDA kernel used in this work, is also not an improvement worth considering. Although at small batches it might be almost 26% faster, robotics training mostly runs on large numbers of parallel environments. In such a scenario, these kernels help only marginally, and the effort required to develop them is considerable. Most important is to write clean code using the functionality of tools like PyTorch, where, instead of using loops, batching can be implemented. This yields the best effort-to-improvement ratio. The only scenario in which implementing such kernels makes sense in robotics is when the pipeline does not need a high-fidelity simulator by itself. For example, in DiffPhysDrone [Zha+25], the authors relied only on their own simulators. And it was developed in CUDA.

Comparing the enhanced code with the inherited one, the full 13-run campaign for the RQ2 took over 28 hours. As the inherited codebase with serial loops scales as $O(B)$, the same campaign would take over a year on the same setup. So, batching the codebase made further research possible.

5.4 RQ2 - The Terrain Information Study

5.4.1 Experiment

In total, 13 training runs were conducted, each with 5000 iterations and 2048 parallel environments. This corresponds to the 240 seconds of simulated time per robot. Since the policy is tested on the curriculum from the Rudin et al. [Rud+22] paper, the aim was to match the simulation numbers to those that Rudin et al. trained on. There, the simulated time was 720 seconds, and batches were both 2048 and 4096 environments [Rud+22]. Here, a smaller batch is used, and the simulation time is shorter, to make the overall runtime of the experiment easier to handle. Also, since the differentiable simulation framework showed faster convergence than the Reinforcement Learning approach on flat terrain [SKS24], the shorter simulation time should be sufficient to show whether the policy can handle rough terrain. The episode time is 20 seconds, and each robot received omnidirectional commands: v_x and v_y uniformly distributed per robot in $[-1; 1]$ m/s, and yaw rate in $[-1; 1]$ rad/s. The command is redrawn after each episode or a reset. This approach is the same as in Rudin et al. [Rud+22].

The five conditions differ only in the path by which the terrain reaches the policy. The `blind_rudin` is the base policy that is tested against hard terrain. Then both `hobs` and `hloss` investigate whether providing the policy with terrain information can improve its locomotion skills. The `height` variant is a statistical probe to check if any of the paths have some influence. The last variant with the depth camera was built to check whether terrain information can improve locomotion in hard terrain, and how much of this information can be captured back in a sim-to-real deployment. All of the variants except for the depth were run with three seeds ($[0, 1, 2]$). The variant with a depth camera was running with a single seed, since it was only trying to reproduce the results of the `hobs` variant.

Mode	Terrain information path	Observation
blind_rudin	none	36D
hobs	187-point elevation scan	223D
hloss	terrain enters the differentiable loss	36D
height	both paths at once	223D
depth	rendered depth image	36D + image

Table 5.4: The five terrain information conditions.

5.4.2 Results

The per-terrain-type maximum terrain level is the primary metric, and the mean over the last 500 iterations is a secondary statistic. The 500 iterations correspond to 24 seconds of simulated time, longer than one 20 seconds episode, so every robot passes through at least one curriculum update. The results of the experiment are presented in Tables 5.5 and 5.6, as well as in Figures 5.3 and 5.4.

Condition	n	Tail-500 mean level	s	Range
blind_rudin	3	0.0011	0.0002	0.0009–0.0012
hobs	3	0.0010	0.0001	0.0009–0.0010
hloss	3	0.0043	0.0004	0.0038–0.0047
height	3	0.0023	0.0005	0.0017–0.0027
depth	1	0.0026	—	—

Table 5.5: Tail-500 mean terrain level. Here n is the number of independent training runs and s their sample standard deviation.

Condition	Smooth slope	Rough slope	Stairs down	Stairs up	Discrete obst.
blind_rudin	0.0033 / 1	0.0000 / 0	0.0000 / 0	0.0000 / 0	0.0008 / 1
hobs	0.0049 / 1	0.0000 / 0	0.0000 / 0	0.0000 / 0	0.0008 / 1
hloss	0.0293 / 2	0.0000 / 0	0.0009 / 1	0.0000 / 0	0.0033 / 1
height	0.0098 / 1	0.0000 / 0	0.0000 / 0	0.0000 / 0	0.0033 / 1
depth	0.0098 / 1	0.0000 / 0	0.0000 / 0	0.0000 / 0	0.0049 / 1

Table 5.6: Per terrain type level: mean averaged over seeds / maximum taken over seeds.

What can be concluded from this experiment is that neither variant of the policy and training was able to handle hard terrain with this set of commands. All runs after the simulation ended up on the easiest level on average. Although some separation can be concluded between the blind policy and the variant with terrain entering as a loss, neither of them could get out of the easiest terrain level on average. So it is taken as a positive signal, though this cannot be confirmed with this data. Only smooth slope moves at all, and it reaches row 2 exactly once, in hloss seed 1.

Also, one limitation bounds all the conclusions. The mean forward velocity cannot be interpreted as a metric, since it is symmetrically drawn. The only informative measure is the velocity tracking error L_v . A standing still policy would score $\mathbb{E}\|v_{\text{cmd}}\|^2 = 0.666$. At the simulation on terrain curriculum, the L_v settles at 34–38% of that value in all thirteen runs. So the robots do walk, but they do not get far. Average distance from the center of the cell

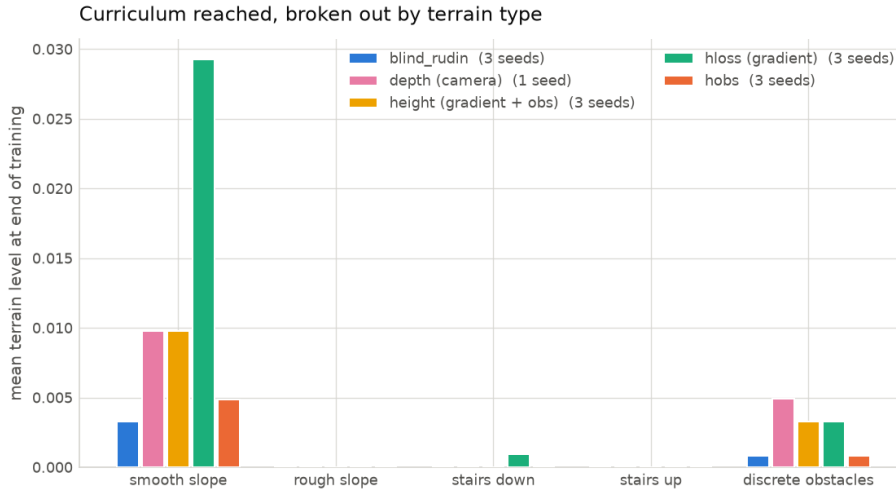


Figure 5.3: Per terrain type level as chart: mean averaged over seeds.

is 0.98 - 1.01 m. If they would never move, the average would be around 0.77 m because of the ± 1 m spawn offset from the cell center. And promotion requires 4 m. So the robots do follow the commands, although they do not travel far enough for reliable terrain promotion and to reliably measure differences between the variants.

A large part of these limitations is the omnidirectional command set. For further investigation, five single-seed runs were run. With 16 robots, 1000 iterations, flat ground, and the same command set. They reached a tracking error of 17% – 25% of the standing still value, against 3% for the same method with a fixed forward command. They also fall 0.18 – 0.38 times per robot per second. At least as often as on the curriculum (0.175 – 0.191), whereas the forward command run ends with no falls at all. So, policy already struggles with omnidirectional commands before any terrain is added. The flat runs are smaller, shorter, and with only one seed. So they show the direction of the effect rather than a matched comparison.

So it is not possible to give a definitive answer on whether propagating terrain gradients improve hard-terrain locomotion. There is some positive signal, but still within the measurement error, and to conduct this experiment, either the curriculum should be adjusted or the policy and training architecture should be changed. The same applies to comparing other variants against each other.

5.5 Perceptual Saliency Diagnostics

To test whether the trained policies use terrain observations, policies from the above locomotion experiment were evaluated. Variants with blind policy, height scan, height scan and gradient, and depth camera were used. The terrain gradient policy was not tested since it does not take terrain as input, just like a normal blind policy, so one baseline probe for this experiment should be enough. During the experiment, policies with the terrain input channel (hobs, height, depth) were using corrupted inputs. Every rollout uses same configuration — 1024 robots, policy from the run with seed 0 from the locomotion experiment, 15000 steps (30 s of simulated time) after a 2000 step warmup, the 10×20 Rudin grid with robots spawned at random levels 0–5 and the omnidirectional command distribution ($v_x, v_y \in [-1, 1]$ m/s, $\omega \in [-1, 1]$ rad/s).

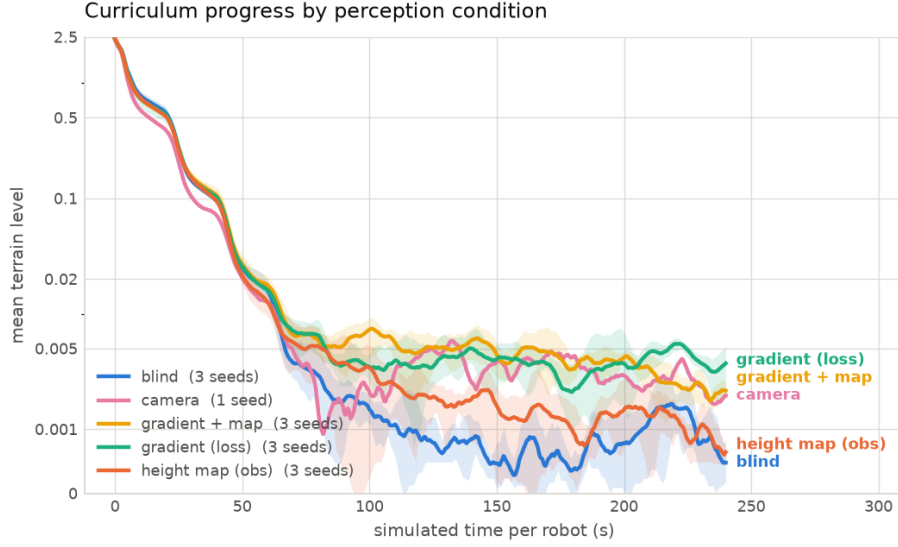


Figure 5.4: Mean terrain level of all 2048 robots over simulated training time per perception conditions. Mean over seeds, 100 iterations rolling mean, band = min/max across seeds.

5.5.1 Comparison Before the Corruption Test

Before applying any corruption input, all three perceptive policies performed worse than the blind baseline. The blind policy reached a mean terrain level of 0.8862, while hobs, height, and depth reached only 0.7593, 0.8027, and 0.7954, respectively. They also had higher fall rates. The differences were much larger than the variation between repeated tests for the same variant. Therefore, the perceptive policies did not show evidence of effectively using their terrain observations, even with clean inputs.

5.5.2 The Corruption Matrix

Two types of input corruption were used. Firstly, a `zero`, which replaced the height map or depth image with zeros, and `shuffle`, which gives each robot the terrain input from another robot. For the depth policy, the shuffle test is the main test because setting depth to zero yields an unrealistic sensor reading rather than a neutral input.

From these results, no conclusions can be drawn, because the changes are inconsistent. For example, zeroing hobs improves performance, whereas shuffling degrades it. Similarly, setting depth to zero also improves performance. However, it does not mean that the policy in such an architecture cannot use the terrain information. The training quickly collapsed to the easiest curriculum level, so the policies had little opportunity to learn to use visual information for locomotion in hard terrain. The evaluation was also difficult because the robots received omnidirectional commands, leading to frequent falls. But these are also command sets that were used in the training.

5.6 Why the Curriculum Collapses

Trained policy	Corruption arm	Mean terrain level	Fall rate	Linear velocity error
blind	None (clean)	0.8862	0.9006	0.2770
hobs	None (clean)	0.7593	0.9249	0.2669
	Zero	0.7910	0.9185	0.2646
	Shuffle	0.7348	0.9369	0.2657
height	None (clean)	0.8027	0.9168	0.2580
	Zero	0.7757	0.9212	0.2585
	Shuffle	0.7977	0.9241	0.2581
depth	None (clean)	0.7954	0.9180	0.2602
	Zero	0.8789	0.9105	0.2667
	Shuffle	0.8120	0.9183	0.2619

Table 5.7: Channel corruption diagnostics results. Fall rate is the fraction of resets caused by a fall among all resets (including those at the end of the 20 seconds episode). Linear velocity error is the gap between the body frame horizontal velocity and the commanded one.

5.6.1 The Shape of the Collapse

The average terrain level does not stay at the same base level. It decreases with the simulation time. By iteration 1000, every run already sits at an average level of 0.022–0.037, so more than three-quarters of the training budget is spent at a floor the population reached within the first fifth of it. The decay begins during the 20th iteration transient. Although it does not influence the performance of the training on the flat ground, on the terrain curriculum, it might degrade itself to a lower level, and then it cannot recover.

5.6.2 The Promotion Arithmetic

A robot is promoted when it travels more than half the length of a terrain cell. As the cell length is $L = 8.0$ m, promotion requires at least 4.0 m of travel. Demotion, on the other hand, occurs if the robot does not cover the distance of the commanded speed times half the episode duration (10 seconds in this case). So, in the great majority of episodes, it is below the promotion threshold, leaving little possibility of staying at the same level. And as they are demoted to the lowest level, since they are, on average, around one meter from the center of the cell, promotion is almost unreachable by this definition. From the measurements, on average, the robot covers only one meter before the reset or the start of the new episode. So the population is moved to level 0 and is trapped there.

5.6.3 Omnidirectional Commands on Flat Ground

To determine whether the failures stem from the terrain or the command set, three settings were compared. Flat ground with a fixed forward command, flat ground with omnidirectional commands, and data from runs from the experiment conducted on the terrain curriculum with omnidirectional commands. The flat runs are much smaller than the curriculum runs: 16 robots against 2048, and 1000 iterations against 5000. Such a setting was chosen because the simulation takes much less time, and with flat ground and a forward command, it is sufficient for the policy to learn locomotion.

The mean base height, number of feet in contact, and average foot height are very similar in all three settings. Nothing indicated that the robots sank into the terrain or that the curriculum was incorrect. Although the minimum foot height on the terrain curriculum can become negative, this is due to how the program measures it, and the value is very small relative to the margin of measurement error. Falls are mainly caused by excessive body tilt. The tilt comes from the omnidirectional command set. For the flat ground with fixed forward command variant, the tilt equals 3.3° , and for the omnidirectional command set, it is 14.0° and 14.9° for the flat ground and terrain, respectively. For the fall rate, there are no falls in the fixed forward variants, while the fall rate per robot per second is very similar across both omnidirectional command set variants. Summing it up, the terrain is not the main cause of the falls. Omnidirectional commands already cause large tilts and frequent falls on flat terrain. The curriculum only adds consequences for this behavior, demoting failing robots to the nearest level.

The comparison is not fully matched because the flat-ground runs use fewer robots, fewer iterations, and only one seed. But these experiments show the direction of the effect rather than its exact magnitude.

5.6.4 Why the Objective Permits Tilt

The loss is strongly dominated by the velocity tracking variable. It ranges from 84% to 86% of the total loss across the terrain curriculum runs. The attitude term (L_{gproj}) on the other hand reaches 1.1 to 1.2 percent. And this is the only term that directly reflects body tilt. The objective does not penalize the falling directly. There is a termination penalty, but it is used only to calculate the episode reward and is not part of the training loss. Since the velocity tracking is the dominant force in the reward, the policy improves the commanded velocity tracking while accepting a significant body lean. The weights of the loss terms were inherited from the original implementation, and they were not changed, since the aim of this project was to test the current policy on the hard terrain and whether the additional terrain information can improve the locomotion skills.

5.7 RQ3 - The Four Interventions

For all interventions, a setup with 1024 robots over 850 iterations (40.8 s of simulated time) is used, with a fixed forward velocity command of 0.5 m/s and a blind policy. This command successfully walks on flat terrain and has the best chance of succeeding on hard terrain with interventions, since it is much easier for the policy to handle. For all interventions, the policy is still blind. No terrain enters it directly, terrain if used, is only used by the foot planner. For the base case run (blind, forward 0.5 m/s, terrain curriculum, 850 iterations, and batch of 1024 robots), the mean terrain at the end of the run is 0.022, with about 1004 of the 1024 robots (98%) on the easiest row. Smooth slope is the only terrain type that climbs at all (mean level 0.18), and stairs up stays at 0. Which means it failed just like all the runs in the perception experiment.

5.7.1 Intervention 1: Differentiable Termination Surrogate

Since the episode termination does not provide a gradient, the loss does not penalize falling. To address this, a soft tilt penalty was introduced:

$$L_{\text{tilt}} = \text{mean}(\max(0, \cos \theta_{\text{on}} - \cos \theta)^2) \quad (5.1)$$

with the penalty activated at $\theta_{\text{on}} = 0.6\text{rad}$ (34°), below the 0.9rad (51.6°) termination threshold. The "mean" refers to the average over all 24 time steps in the iteration and all environments.

The implementation was tested against runs without the penalty, and the weights chosen for this term were both $w = 16$ and $w = 48$. This corresponds to approximately 5% and 15% of the objective during early training. All runs were tested on the terrain curriculum, with a fixed forward speed command of 0.5 m/s. All tests were run on a batch of 1024 robots over 850 iterations.

The term remained active during the training. However, the penalty had no meaningful effect on the training outcome. Regardless of the weight assigned to L_{tilt} .

Metric	w0	w16	w48
<i>Tilt term</i>			
Weight w	0	16	48
$\mathcal{L}_{\text{tilt}}$ (last 200)	-	0.002	0.002
$w \cdot \mathcal{L}_{\text{tilt}}$ (last 200)	0	0.032	0.089
Total loss (last 200)	0.974	0.989	1.041
Share of total loss, last 200	0	3.2 %	8.5 %
<i>Curriculum</i>			
Final level	0.0218	0.0381	0.0205
Robots on row 0 (of 1024)	1004	992	1005
Smooth slope	0.175	0.233	0.136
Stairs down	0.009	0.017	0.003
Obstacles	0.005	0.044	0.029
Stairs up	0	0	0
Rough slope	0	0	0

Table 5.8: Results of the Intervention 1

5.7.2 Intervention 2: Terrain Aware Swing Planner

In the inherited method, the swing planner sets the foot's landing height based on the previous foot contact. This assumption is correct on the flat terrain, but can be problematic on stairs, for example. This intervention takes the landing height from the terrain scan. Additionally, the system can increase the swing apex if higher terrain is detected.

Three variants were tested. The original planner, terrain-aware landing height, and terrain-aware landing height combined with the terrain-aware swing apex. The implementation was verified to result in unchanged behavior when both options are disabled.

The terrain-aware targets had no meaningful effect on policy learning in the hard terrain. Regardless of whether only landing height or swing apex adjustments are considered. The variant with the terrain-aware landing height did the best out of the three variants, but only marginally. So it cannot be decided if it really helps in this curriculum.

Metric	Blind baseline	Landing height	Landing height + apex
Final level	0.0218	0.0391	0.0225
Robots on row 0 (of 1024)	1004	985	1005
Smooth slope	0.175	0.291	0.146
Rough slope	0	0	0
Stairs down	0.009	0.008	0.008
Stairs up	0	0	0
Obstacles	0.005	0.034	0.025

Table 5.9: Results of the Intervention 2

5.7.3 Intervention 3: Explicit Foothold Residual

As the next step of Intervention 2, the policy’s output was expanded from 12 values to 16 (20 with the lateral corrections). With the scan of the terrain (corresponding to the input of blind 36D and 187D height scan), policy got a possibility to adjust each planned foothold by up to ± 0.10 m, either in the x-axis only or both in x and y axis directions. New foothold quality loss L_{fq} encourages the policy to place the feet on stable terrain instead of near edges or gaps. Additionally, there is another term L_{fres} that keeps the correction near zero unless the terrain justifies moving the foot. The L_{foot} is detached from this channel, so the policy must move the foot to a more optimal placement rather than reusing the current foothold.

The tests showed that the policy did not use the foothold corrections. The mean magnitude at the end of the runs was only 0.23 mm per axis, corresponding to around 0.2% of the possible range. Both variants with sagittal only and sagittal with lateral corrections behaved similarly. The main reason was that the L_{fq} gradient nearly zeroed out after the robots were downgraded to the easiest terrain level. Since the L_{foot} was detached, the L_{fres} became the main remaining gradient, pushing the steps towards the uncorrected values. The assumption is that the possible outputs of the policy were not the problem, but simply a lack of a useful terrain gradient.

5.7.4 Intervention 4: Extended Gradient Horizon

As the baseline simulation iteration takes 24 steps, corresponding to 48 ms, this is only 7.7% of the gait cycle period. To test if the horizon is too short, the window was doubled to 48 steps and 96 ms, respectively. Two settings were tested, with 425 iterations, matching the simulated time of the baseline run, and 850 iterations. All tests are conducted under a blind policy on a terrain curriculum with 1024 robots and a fixed forward-velocity command of 0.5 m/s.

Doubling the gradient horizon did not improve locomotion. The differences are within the margin of statistical error. The curriculum progression was also similar to the baseline, although the longer horizon slightly delayed the collapse. Fall rates per simulated second remained almost unchanged. These results show that prolonging the gradient horizon does not help the policy to handle hard terrain.

Summing up all the interventions, research question number 3 cannot be answered directly. Given the above results, it is hard to determine whether the aforementioned limits are incorrectly assessed or whether the solutions to them are not correct.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

This thesis extended the method of Song et al. [SKS24]. Song et al. pointed out that vision-based control is a direction for future work on their method, and this thesis tried to implement that, also validating whether this architecture can handle a hard-terrain curriculum. The codebase was vectorized and accelerated, enabling further research and more simulations in a reasonable time. The terrain curriculum of `legged_gym` [Rud+22] was ported, and terrain information was given to the policy as a height scan, a loss gradient, and a depth image.

The first research question of this work asked what determines throughput and whether, under such research conditions, one needs to implement a custom CUDA kernel to achieve good training speeds. It can be concluded that, as long as a high-fidelity simulator is involved, there is no need to write custom accelerators. Most important is to make sure that the batching of the environments is used correctly, according to the possibilities of the tools/programming languages used, such as PyTorch. Wrappers over low-level programming approaches, such as using PyTorch GPU support instead of CUDA, are perfectly fine. As long as they are utilized properly. Without using simulators like NVIDIA Isaac Gym, writing custom kernels might help achieve additional performance gains. Without proper batching, conducting experiments in robotics would be very hard, as it takes a lot of simulation effort.

The next question asked to what extent terrain gradients improve hard terrain locomotion. In all 13 runs, the curriculum collapsed to the easiest level, and no robot on the terrain "stairs up" ended above level 0. With terrain in the loss, the level stayed slightly above the blind baseline in all three seeds, but this is only a small positive signal. The perceptive policies also showed no sign of using their terrain input. The falls behind the collapse are not mainly caused by the terrain. The omnidirectional commands already make the robot tilt and fall on flat ground. In the curriculum, almost every fall results in a demotion, and the loss, dominated by velocity tracking, does not penalize falling. So this question cannot be answered definitively, since the training formulation, not the terrain signal, limits the result. It might be that, to test this research question, the architecture of this differentiable simulation approach would need to be redesigned. This thesis did not arrive at a definitive solution for what would need to be changed to make this experiment more informative. In this form, the policy was tested and found unable to handle a terrain curriculum, on which a massively parallel Reinforcement Learning framework was trained successfully [Rud+22].

The third research question aimed to find out which structural properties limit the use of the gradient. Three limits were identified. No policy output that acts on the terrain, falls happening outside of the gradient window, and a short gradient window. None of the four interventions against them successfully prevented the policies from collapsing. Although they were tested on the easier command set, to give more chances to the curriculum, so that it

does not fall. A potential explanation for that is the case of the foothold residual. The policy reduced its foothold correction to almost zero because, on the flat ground of the collapsed curriculum, the loss of foothold quality had no gradient left. A policy can only learn to use terrain on hard terrain, but it never stays there long enough to learn it. Therefore, the third research question cannot be answered either. It is hard to measure whether the assumptions were incorrect, whether the solutions are wrong, or whether the curriculum is simply not suited to this policy.

Overall, terrain information alone does not bring the method from flat ground to hard terrain. Song et al. [SKS24] noted that differentiable simulation depends on the information in its objective, while reinforcement learning can easily add termination penalties against falling. This thesis showed what this means on hard terrain. Before a terrain signal can help, the training must first keep the robot upright.

6.2 Future Work

Firstly, the training setup should let the robots survive long enough to reach hard terrain. As the collapse starts with a large tilt early in the training, it could be investigated whether starting the curriculum from a policy that already walks on flat ground could bring any benefits. This effect is small with a fixed forward command, but becomes much larger with omnidirectional commands, suggesting that the main problem comes from the command distribution rather than the terrain itself. The loss function also contributes to this problem. Velocity tracking accounts for most of the total loss, while the attitude terms participation is marginal. In addition, falling is not directly penalized because the termination penalty is being used only for the logged reward and not for the training loss. Maybe a better approach would be to first train the policy on flat terrain and gradually introduce lateral and yaw commands once stable tracking is achieved. The objective could also be adjusted to place greater emphasis on maintaining a stable body orientation. Also, the curriculum’s suitability for such a differentiable framework should be further investigated. If not, what could be changed? For the purposes of this thesis, it was simply ported as the state-of-the-art training curriculum used for RL architectures. It does not account for the much shorter rollout used here. Maybe a fall should not always cost a level degradation. The distance used for promotions could be adjusted to what robots can realistically travel. Once the curriculum is stable, the different ways of providing terrain information can be evaluated more reliably. Also, the proposed interventions in this work could be retested to check whether they add value.

Secondly, developing a solution to provide objective information about what happens after the gradient window could be another research direction. The gradient norms were much smaller than the clipping threshold, so gradient clipping did not affect the training. This suggests that the problem is not the length of the gradient window, but that the window does not contain enough information about future falls. In the framework of SHAC [Xu+22], a learned value function was successfully utilized to mark states close to the fall as expensive. Luo et al. [Luo+24] used a similar critic for perceptive quadruped control in differentiable simulation. A possible solution could combine three changes: discount the loss over time instead of averaging it, add a terminal value at the end of the window, and include the termination penalty in the training loss when a fall occurs. This would give the policy a stronger signal that states leading to a fall are undesirable.

Thirdly, a gait schedule driven by contact events could be an interesting approach to explore within this framework. The current gait is controlled only by the phase clock. The robot switches between stance and swing according to the planned phase, while the measured foot contacts are not used to change the gait. As a result, on the `legged_gym`’s curriculum,

the lowest foot can reach below the terrain surface, for example, when the foot hits a stair edge while following its predefined trajectory. To improve it, an early contact could switch the leg to stance, while a late contact could make the leg move down until it finds the ground. If the foot collides with an obstacle during the swing phase, it could also be retracted. Such an approach was used on the MIT Cheetah 3 platform [Ble+18b], allowing it to handle unstructured terrain. This approach does not require terrain perception. It should therefore be introduced before further perception experiments. To implement it, the gait phase must be stored separately for each leg. The switch itself would be discrete, but the calculations after the switch remain differentiable. Therefore, gradients can still flow through the selected branch, similar to the existing stance mask.

Finally, no policy was tested on the real Unitree Go2, so sim-to-real transfer remains open.

6.3 Limitations

Two key design decisions constrain the conclusions of this thesis and should be outlined. First, the omnidirectional command distribution of `legged_gym` [Rud+22] remained unchanged. This choice was deliberate so that the results could be compared with those achieved by Rudin et al. in their work [Rud+22]. However, following the collapse of the curriculum, this comparison lost its purpose. Analysis of the results in this project identified the commands as the main source of the falls. A restricted or gradually widened command set was not evaluated on the terrain curriculum, although this is the most prominent next step. Second, the loss weights inherited from the baseline implementation [Zha26] were left unchanged in all experiments. The objective was to assess the applicability of the method proposed by Song et al. [SKS24] in its base form to hard terrain, rather than to evaluate a modified method. Retuning the weights without a principled procedure would have amounted to an arbitrary search and would have confounded the effect of the terrain signal with the effect of the reweighting. Therefore, this thesis does not demonstrate that the method cannot handle hard terrain. It shows that it cannot do so under the inherited objective and command distribution designed for reinforcement learning.

Appendix A

Code and Results

Code described in this thesis is accessible in a public repository at <https://github.com/kubajura3/BachelorThesis>.

All of the results and logs from the runs used for the experiments as well as the flame-graphs from the code profiling can be found under the folder `results`. Documentation is also provided, so that the reproduction of the results is possible.

Bibliography

- [Aga+22] Agarwal, A., Kumar, A., Malik, J., and Pathak, D. *Legged Locomotion in Challenging Terrains using Egocentric Vision*. 2022. arXiv: 2211.07638 [cs.R0]. URL: <https://arxiv.org/abs/2211.07638>.
- [Bay+18] Baydin, A. G., Pearlmutter, B. A., Radul, A. A., and Siskind, J. M. *Automatic differentiation in machine learning: a survey*. 2018. arXiv: 1502.05767 [cs.SC]. URL: <https://arxiv.org/abs/1502.05767>.
- [Bel+18] Bellicoso, C. D., Jenelten, F., Gehring, C., and Hutter, M. “Dynamic Locomotion Through Online Nonlinear Motion Optimization for Quadrupedal Robots”. In: *IEEE Robotics and Automation Letters* 3.3 (2018), pp. 2261–2268. DOI: 10.1109/LRA.2018.2794620.
- [Ble+18a] Bledt, G., Powell, M. J., Katz, B., Di Carlo, J., Wensing, P. M., and Kim, S. “MIT Cheetah 3: Design and Control of a Robust, Dynamic Quadruped Robot”. In: *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2018, pp. 2245–2252. DOI: 10.1109/IROS.2018.8593885.
- [Ble+18b] Bledt, G., Wensing, P. M., Ingersoll, S., and Kim, S. “Contact Model Fusion for Event-Based Locomotion in Unstructured Terrains”. In: *2018 IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 4399–4406. DOI: 10.1109/ICRA.2018.8460904.
- [Che+23] Cheng, X., Shi, K., Agarwal, A., and Pathak, D. *Extreme Parkour with Legged Robots*. 2023. arXiv: 2309.14341 [cs.R0]. URL: <https://arxiv.org/abs/2309.14341>.
- [Deg+18] Degraeve, J., Hermans, M., Dambre, J., and wyffels, F. *A Differentiable Physics Engine for Deep Learning in Robotics*. 2018. arXiv: 1611.01652 [cs.NE]. URL: <https://arxiv.org/abs/1611.01652>.
- [Di +18] Di Carlo, J., Wensing, P. M., Katz, B., Bledt, G., and Kim, S. “Dynamic Locomotion in the MIT Cheetah 3 Through Convex Model-Predictive Control”. In: *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2018, pp. 1–9. DOI: 10.1109/IROS.2018.8594448.
- [Don+26] Dong, Y., Ma, J., Lu, Y., Zhang, J., Li, W., Chen, Y., Zhang, T., Chen, X., Yu, Z., and Lu, P. “MGDP: Mastering a Generalized Depth Perception Model for Quadruped Locomotion”. In: *Advanced Science* 13.34 (2026), e24345. DOI: 10.1002/advs.202524345. eprint: <https://advanced.onlinelibrary.wiley.com/doi/pdf/10.1002/advs.202524345>. URL: <https://advanced.onlinelibrary.wiley.com/doi/abs/10.1002/advs.202524345>.
- [Fan+18] Fankhauser, P., Bjelonic, M., Dario Bellicoso, C., Miki, T., and Hutter, M. “Robust Rough-Terrain Locomotion with a Quadrupedal Robot”. In: *2018 IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 5761–5768. DOI: 10.1109/ICRA.2018.8460731.

- [Fre26] Frederickson, B. *py-spy: Sampling profiler for Python programs*. <https://github.com/benfred/py-spy>. Accessed: 2026-09-09. 2026.
- [Fre+21] Freeman, C. D., Frey, E., Raichuk, A., Girgin, S., Mordatch, I., and Bachem, O. *Brax – A Differentiable Physics Engine for Large Scale Rigid Body Simulation*. 2021. arXiv: 2106.13281 [cs.R0]. URL: <https://arxiv.org/abs/2106.13281>.
- [HSS25] Heeg, J., Song, Y., and Scaramuzza, D. “Learning Quadrotor Control from Visual Features Using Differentiable Simulation”. In: *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, May 2025, pp. 4033–4039. DOI: 10.1109/icra55743.2025.11128641. URL: <http://dx.doi.org/10.1109/ICRA55743.2025.11128641>.
- [How+25] Howell, T. A., Le Cleac’h, S., Brüdigam, J., Chen, Q., Sun, J., Kolter, J. Z., Schwager, M., and Manchester, Z. *Dojo: A Differentiable Physics Engine for Robotics*. 2025. arXiv: 2203.00806 [cs.R0]. URL: <https://arxiv.org/abs/2203.00806>.
- [Hut+16] Hutter, M., Gehring, C., Jud, D., Lauber, A., Bellicoso, C. D., Tsounis, V., Hwangbo, J., Bodie, K., Fankhauser, P., Bloesch, M., Diethelm, R., Bachmann, S., Melzer, A., and Hoepflinger, M. “ANYmal - a highly mobile and dynamic quadrupedal robot”. In: *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2016, pp. 38–44. DOI: 10.1109/IROS.2016.7758092.
- [Hwa+19] Hwangbo, J., Lee, J., Dosovitskiy, A., Bellicoso, D., Tsounis, V., Koltun, V., and Hutter, M. “Learning agile and dynamic motor skills for legged robots”. In: *Science Robotics* 4.26 (Jan. 2019). ISSN: 2470-9476. DOI: 10.1126/scirobotics.aau5872. URL: <http://dx.doi.org/10.1126/scirobotics.aau5872>.
- [JRM26] Jakob, W., Rhineland, J., and Moldovan, D. *pybind11: Seamless operability between C++11 and Python*. <https://github.com/pybind/pybind11>. Version 3.0.4, accessed: 2026-09-06. 2026.
- [Jen+22] Jenelten, F., Grandia, R., Farshidian, F., and Hutter, M. “TAMOLS: Terrain-Aware Motion Optimization for Legged Systems”. In: *IEEE Transactions on Robotics* 38.6 (Dec. 2022), pp. 3395–3413. ISSN: 1941-0468. DOI: 10.1109/tro.2022.3186804. URL: <http://dx.doi.org/10.1109/TRO.2022.3186804>.
- [Kim+19] Kim, D., Carlo, J. D., Katz, B., Bledt, G., and Kim, S. *Highly Dynamic Quadruped Locomotion via Whole-Body Impulse Control and Model Predictive Control*. 2019. arXiv: 1909.06586 [cs.R0]. URL: <https://arxiv.org/abs/1909.06586>.
- [Kui+16] Kuindersma, S., Deits, R., Fallon, M., Valenzuela, A., Dai, H., Permenter, F., Koolen, T., Marion, P., and Tedrake, R. “Optimization-based locomotion planning, estimation, and control design for the atlas humanoid robot”. In: *Autonomous Robots* 40.3 (Mar. 2016), pp. 429–455. DOI: 10.1007/s10514-015-9479-3. URL: <https://doi.org/10.1007/s10514-015-9479-3>.
- [Kum+21] Kumar, A., Fu, Z., Pathak, D., and Malik, J. *RMA: Rapid Motor Adaptation for Legged Robots*. 2021. arXiv: 2107.04034 [cs.LG]. URL: <https://arxiv.org/abs/2107.04034>.
- [Lee+20] Lee, J., Hwangbo, J., Wellhausen, L., Koltun, V., and Hutter, M. “Learning quadrupedal locomotion over challenging terrain”. In: *Science Robotics* 5.47 (Oct. 2020). ISSN: 2470-9476. DOI: 10.1126/scirobotics.abc5986. URL: <http://dx.doi.org/10.1126/scirobotics.abc5986>.

- [Loq+21] Loquercio, A., Kaufmann, E., Ranftl, R., Müller, M., Koltun, V., and Scaramuzza, D. “Learning high-speed flight in the wild”. In: *Science Robotics* 6.59 (Oct. 2021). ISSN: 2470-9476. DOI: 10.1126/scirobotics.abg5810. URL: <http://dx.doi.org/10.1126/scirobotics.abg5810>.
- [LH19] Loshchilov, I. and Hutter, F. *Decoupled Weight Decay Regularization*. 2019. arXiv: 1711.05101 [cs.LG]. URL: <https://arxiv.org/abs/1711.05101>.
- [Luo+24] Luo, J. Y., Song, Y., Klemm, V., Shi, F., Scaramuzza, D., and Hutter, M. *Residual Policy Learning for Perceptive Quadruped Control Using Differentiable Simulation*. 2024. arXiv: 2410.03076 [cs.R0]. URL: <https://arxiv.org/abs/2410.03076>.
- [Mak+21] Makoviychuk, V., Wawrzyniak, L., Guo, Y., Lu, M., Storey, K., Macklin, M., Hoeller, D., Rudin, N., Allshire, A., Handa, A., and State, G. *Isaac Gym: High Performance GPU-Based Physics Simulation For Robot Learning*. 2021. arXiv: 2108.10470 [cs.R0]. URL: <https://arxiv.org/abs/2108.10470>.
- [Mar+22] Margolis, G. B., Yang, G., Paigwar, K., Chen, T., and Agrawal, P. *Rapid Locomotion via Reinforcement Learning*. 2022. arXiv: 2205.02824 [cs.R0]. URL: <https://arxiv.org/abs/2205.02824>.
- [Met+22] Metz, L., Freeman, C. D., Schoenholz, S. S., and Kachman, T. *Gradients are Not All You Need*. 2022. arXiv: 2111.05803 [cs.LG]. URL: <https://arxiv.org/abs/2111.05803>.
- [Mik+22] Miki, T., Lee, J., Hwangbo, J., Wellhausen, L., Koltun, V., and Hutter, M. “Learning robust perceptive locomotion for quadrupedal robots in the wild”. In: *Science Robotics* 7.62 (Jan. 2022). ISSN: 2470-9476. DOI: 10.1126/scirobotics.abk2822. URL: <http://dx.doi.org/10.1126/scirobotics.abk2822>.
- [Mor+21] Mora, M. A. Z., Peychev, M., Ha, S., Vechev, M., and Coros, S. “PODS: Policy Optimization via Differentiable Simulation”. In: *Proceedings of the 38th International Conference on Machine Learning*. Ed. by Meila, M. and Zhang, T. Vol. 139. Proceedings of Machine Learning Research. PMLR, 18–24 Jul 2021, pp. 7805–7817. URL: <https://proceedings.mlr.press/v139/mora21a.html>.
- [New+24] Newbury, R., Collins, J., He, K., Pan, J., Posner, I., Howard, D., and Cosgun, A. *A Review of Differentiable Simulators*. 2024. arXiv: 2407.05560 [cs.R0]. URL: <https://arxiv.org/abs/2407.05560>.
- [NVI26] NVIDIA Corporation. *Warp Documentation*. <https://nvidia.github.io/warp/stable/>. Version 1.16.0, accessed 2026-08-17. 2026.
- [OGL13] Orin, D. E., Goswami, A., and Lee, S.-H. “Centroidal dynamics of a humanoid robot”. In: *Autonomous Robots* 35.2-3 (Oct. 2013), pp. 161–176. DOI: 10.1007/s10514-013-9341-4. URL: <https://doi.org/10.1007/s10514-013-9341-4>.
- [Pen+18a] Peng, X. B., Abbeel, P., Levine, S., and Panne, M. van de. “DeepMimic: example-guided deep reinforcement learning of physics-based character skills”. In: *ACM Transactions on Graphics* 37.4 (July 2018), pp. 1–14. ISSN: 1557-7368. DOI: 10.1145/3197517.3201311. URL: <http://dx.doi.org/10.1145/3197517.3201311>.
- [Pen+18b] Peng, X. B., Andrychowicz, M., Zaremba, W., and Abbeel, P. “Sim-to-Real Transfer of Robotic Control with Dynamics Randomization”. In: *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, May 2018, pp. 3803–3810. DOI: 10.1109/icra.2018.8460528. URL: <http://dx.doi.org/10.1109/ICRA.2018.8460528>.

- [Pes+25] Pestova, E., Osokin, I., Belov, D., and Osinenko, P. *Adaptive MPC-based quadrupedal robot control under periodic disturbances*. 2025. arXiv: 2505.12361 [cs.R0]. URL: <https://arxiv.org/abs/2505.12361>.
- [Rai86] Raibert, M. H. “Legged robots”. In: *Commun. ACM* 29.6 (June 1986), pp. 499–514. ISSN: 0001-0782. DOI: 10.1145/5948.5950. URL: <https://doi.org/10.1145/5948.5950>.
- [Rai+83] Raibert, M. H., Brown, H. B., Chepponis, M., Hastings, E., Koechling, J., Murphy, K. N., Murthy, S. S., and Stentz, A. (*Dynamically Stable Legged Locomotion Progress Report: October 1982 - October 1983*. Tech. rep. CMU-RI-TR-83-20. Pittsburgh, PA: Carnegie Mellon University, Dec. 1983.
- [Rec18] Recht, B. *A Tour of Reinforcement Learning: The View from Continuous Control*. 2018. arXiv: 1806.09460 [math.OC]. URL: <https://arxiv.org/abs/1806.09460>.
- [Ren+23] Ren, J., Yu, C., Chen, S., Ma, X., Pan, L., and Liu, Z. *DiffMimic: Efficient Motion Mimicking with Differentiable Physics*. 2023. arXiv: 2304.03274 [cs.CV]. URL: <https://arxiv.org/abs/2304.03274>.
- [RGB11] Ross, S., Gordon, G. J., and Bagnell, J. A. *A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning*. 2011. arXiv: 1011.0686 [cs.LG]. URL: <https://arxiv.org/abs/1011.0686>.
- [Rud+22] Rudin, N., Hoeller, D., Reist, P., and Hutter, M. *Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning*. 2022. arXiv: 2109.11978 [cs.R0]. URL: <https://arxiv.org/abs/2109.11978>.
- [Sch+17a] Schulman, J., Levine, S., Moritz, P., Jordan, M. I., and Abbeel, P. *Trust Region Policy Optimization*. 2017. arXiv: 1502.05477 [cs.LG]. URL: <https://arxiv.org/abs/1502.05477>.
- [Sch+17b] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. *Proximal Policy Optimization Algorithms*. 2017. arXiv: 1707.06347 [cs.LG]. URL: <https://arxiv.org/abs/1707.06347>.
- [SKS24] Song, Y., Kim, S., and Scaramuzza, D. *Learning Quadruped Locomotion Using Differentiable Simulation*. 2024. arXiv: 2403.14864 [cs.R0]. URL: <https://arxiv.org/abs/2403.14864>.
- [Son+23] Song, Y., Romero, A., Müller, M., Koltun, V., and Scaramuzza, D. “Reaching the limit in autonomous racing: Optimal control versus reinforcement learning”. In: *Science Robotics* 8.82 (Sept. 2023). ISSN: 2470-9476. DOI: 10.1126/scirobotics.adg1462. URL: <http://dx.doi.org/10.1126/scirobotics.adg1462>.
- [SS24] Song, Y. and Scaramuzza, D. “Learning Robot Control: From Reinforcement Learning to Differentiable Simulation”. In: *Workshop on Embodiment-Aware Robot Learning*. 2024. URL: <https://openreview.net/forum?id=3on357KP7e>.
- [Suh+22] Suh, H. J. T., Simchowicz, M., Zhang, K., and Tedrake, R. *Do Differentiable Simulators Give Better Policy Gradients?* 2022. arXiv: 2202.00817 [cs.LG]. URL: <https://arxiv.org/abs/2202.00817>.
- [Tan+18] Tan, J., Zhang, T., Coumans, E., Iscen, A., Bai, Y., Hafner, D., Bohez, S., and Vanhoucke, V. *Sim-to-Real: Learning Agile Locomotion For Quadruped Robots*. 2018. arXiv: 1804.10332 [cs.R0]. URL: <https://arxiv.org/abs/1804.10332>.
- [Tob+17] Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., and Abbeel, P. *Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World*. 2017. arXiv: 1703.06907 [cs.R0]. URL: <https://arxiv.org/abs/1703.06907>.

- [Uni26a] Unitree Robotics. *Unitree Developer Documentation*. Accessed: 2026-08-17. Unitree Robotics. 2026. URL: <https://support.unitree.com/home/en/developer>.
- [Uni26b] Unitree Robotics. *Unitree Go2 URDF Description: go2_description.urdf from the unitree_ros repository*. https://github.com/unitreerobotics/unitree_ros/blob/master/robots/go2_description/urdf/go2_description.urdf. Accessed: 2026-09-08. 2026.
- [Wer90] Werbos, P. J. “Backpropagation through time: what it does and how to do it”. In: *Proceedings of the IEEE* 78.10 (1990), pp. 1550–1560. DOI: 10.1109/5.58337.
- [Wer+21] Werling, K., Omens, D., Lee, J., Exarchos, I., and Liu, C. K. *Fast and Feature-Complete Differentiable Physics for Articulated Rigid Bodies with Contact*. 2021. arXiv: 2103.16021 [cs.R0]. URL: <https://arxiv.org/abs/2103.16021>.
- [Win+18] Winkler, A. W., Bellicoso, C. D., Hutter, M., and Buchli, J. “Gait and Trajectory Optimization for Legged Systems Through Phase-Based End-Effector Parameterization”. In: *IEEE Robotics and Automation Letters* 3.3 (2018), pp. 1560–1567. DOI: 10.1109/LRA.2018.2798285.
- [Xu+22] Xu, J., Makoviychuk, V., Narang, Y., Ramos, F., Matusik, W., Garg, A., and Macklin, M. *Accelerated Policy Learning with Parallel Differentiable Simulation*. 2022. arXiv: 2204.07137 [cs.LG]. URL: <https://arxiv.org/abs/2204.07137>.
- [Yan+21] Yang, Y., Zhang, T., Coumans, E., Tan, J., and Boots, B. *Fast and Efficient Locomotion via Learned Gait Transitions*. 2021. arXiv: 2104.04644 [cs.R0]. URL: <https://arxiv.org/abs/2104.04644>.
- [Ye+22] Ye, L., Wang, X., Liu, H., and Liang, B. *The Simplest Balance Controller for Dynamic Walking*. 2022. arXiv: 2211.06223 [cs.R0]. URL: <https://arxiv.org/abs/2211.06223>.
- [Zha26] Zhang, R. *Quadruped Robot Gait Training System: project aiming to reproduce Yunlong Song’s paper: "Learning Quadruped Locomotion Using Differentiable Simulation"*. <https://github.com/RonGenZ/RonGenZ>. Accessed: 2026-04-19. 2026.
- [Zha+25] Zhang, Y., Hu, Y., Song, Y., Zhang, D., Yin, X., and Scaramuzza, D. “Learning vision-based agile flight via differentiable physics”. In: *Nature Machine Intelligence* 7 (June 2025), pp. 954–966. DOI: 10.1038/s42256-025-01048-0. URL: <https://doi.org/10.1038/s42256-025-01048-0>.
- [Zhu+23] Zhuang, Z., Fu, Z., Wang, J., Atkeson, C., Schwertfeger, S., Finn, C., and Zhao, H. *Robot Parkour Learning*. 2023. arXiv: 2309.05665 [cs.R0]. URL: <https://arxiv.org/abs/2309.05665>.